

[Euron] 머신러닝 스터디 5주차

[통계](#) [수정](#) [삭제](#)

조운서 · 방금 전

❤️ 0

(Euron) 머신러닝

▼ 목록 보기

4/4



교재: 파이썬 머신러닝 완벽 가이드 (권철민)

캐글 노트북1: Decision Tree And Random Forest Classifier Models <https://www.kaggle.com/code/ihsncnkz/decision-tree-and-random-forest-classifier-models#Description>캐글 노트북2: Beginner Friendly CatBoost with Optuna <https://www.kaggle.com/code/kaanboke/beginner-friendly-catboost-with-optuna>

제 4장. 분류

09. 분류 실습 - 캐글 산탄데르 고객 만족 예측

- 캐글의 산탄데르 고객 만족(Santander Customer Satisfaction) 데이터 세트에 대해 고객 만족 여부를 XGBoost와 LightGBM을 활용해 예측
- 클래스 레이블 명: 'TARGET'. 1이면 불만을 가진 고객, 0이면 만족한 고객

1. 데이터 전처리

(1) 데이터 분포 확인

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib
import warnings

warnings.filterwarnings('ignore')
cust_df = pd.read_csv('./train_santander.csv', encoding='latin-1')
print('dataset shape:', cust_df.shape)
cust_df.head(3)
```

dataset shape: (76020, 371)

	ID	var3	var15	imp_ent_var16_ult1	imp_op_var39_comer_ult1	imp_op_var39_comer_ult3	imp_op_var40_comer_ult1
0	1	2	23	0.0	0.0	0.0	0.0
1	3	2	34	0.0	0.0	0.0	0.0
2	4	2	23	0.0	0.0	0.0	0.0

3 rows × 371 columns

cust_df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 76020 entries, 0 to 76019
Columns: 371 entries, ID to TARGET
dtypes: float64(111), int64(260)
memory usage: 215.2 MB
```

- 클래스 값 칼럼을 포함한 피처가 371개 존재
- 111개의 피처가 float 형, 260개의 피처가 int 형 -> 모든 피처 숫자형
- Null값은 존재X

```
print(cust_df['TARGET'].value_counts())
unsatisfied_cnt = cust_df[cust_df['TARGET'] == 1].TARGET.count()
total_cnt = cust_df.TARGET.count()
print(f'unsatisfied 비율은 {(unsatisfied_cnt / total_cnt):.2f}')
```

```
TARGET
0    73012
1     3008
Name: count, dtype: int64
unsatisfied 비율은 0.04
```

cust_df.describe()

	ID	var3	var15	imp_ent_var16_ult1	imp_op_var39_comer_ult1	imp_op_var39_comer_ult3
count	76020.000000	76020.000000	76020.000000	76020.000000	76020.000000	76020.000000
mean	75964.050723	-1523.199277	33.212865	86.208265	72.363067	119.529632
std	43781.947379	39033.462364	12.956486	1614.757313	339.315831	546.266294
min	1.000000	-999999.000000	5.000000	0.000000	0.000000	0.000000
25%	38104.750000	2.000000	23.000000	0.000000	0.000000	0.000000
50%	76043.000000	2.000000	28.000000	0.000000	0.000000	0.000000
75%	113748.750000	2.000000	40.000000	0.000000	0.000000	0.000000
max	151838.000000	238.000000	105.000000	210000.000000	12888.030000	21024.810000

8 rows × 7 columns

- 불만족 고객의 비율은 4%
- var3 칼럼의 경우, Q1, Q2, Q3가 2인데 min 값은 -999999

(2) 전처리

```
cust_df['var3'].replace(-999999, 2, inplace=True)
cust_df.drop('ID', axis=1, inplace=True)

cust_df.describe()
```

	var3	var15	imp_ent_var16_ult1	imp_op_var39_comer_ult1	imp_op_var39_comer_ult3	imp_op_var40
count	76020.000000	76020.000000	76020.000000	76020.000000	76020.000000	
mean	2.716483	33.212865	86.208265	72.363067	119.529632	
std	9.447971	12.956486	1614.757313	339.315831	546.266294	
min	0.000000	5.000000	0.000000	0.000000	0.000000	
25%	2.000000	23.000000	0.000000	0.000000	0.000000	
50%	2.000000	28.000000	0.000000	0.000000	0.000000	
75%	2.000000	40.000000	0.000000	0.000000	0.000000	
max	238.000000	105.000000	210000.000000	12888.030000	21024.810000	

8 rows × 370 columns

- var3의 편차가 심하므로 -999999을 2로 변환
- ID 피처는 단순 식별자이므로 삭제

(3) 데이터 쪼개기

```
X_features = cust_df.iloc[:, :-1]
y_labels = cust_df.iloc[:, -1]
print(f'피처 데이터 shape:{X_features.shape}')
```

피처 데이터 shape:(76020, 369)

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X_features, y_labels,
                                                    test_size=0.2, random_state=0)

train_cnt = y_train.count()
test_cnt = y_test.count()
print(f'학습 세트 Shape: {X_train.shape}, 테스트 세트 Shape: {X_test.shape}')
print(y_train.value_counts()/train_cnt)
print('\n 테스트 세트 레이블 값 분포 비율')
print(y_test.value_counts()/test_cnt)
```

학습 세트 Shape: (60816, 369), 테스트 세트 Shape: (15204, 369)

TARGET

0 0.960964

1 0.039036

Name: count, dtype: float64

테스트 세트 레이블 값 분포 비율

TARGET

0 0.9583

1 0.0417

Name: count, dtype: float64

```
# X_train, y_train을 다시 학습과 검증 데이터 세트로 분리
```

```
X_tr, X_val, y_tr, y_val = train_test_split(X_train, y_train, test_size=0.3, random_state=0)
```

- 학습과 테스트 데이터 세트 모두 TARGET 값의 분포가 원본 데이터와 유사하게 전체 데이터의 4% 정도의 불만족 값으로 만들어짐

2. XGBoost 모델 학습과 하이퍼 파라미터 튜닝

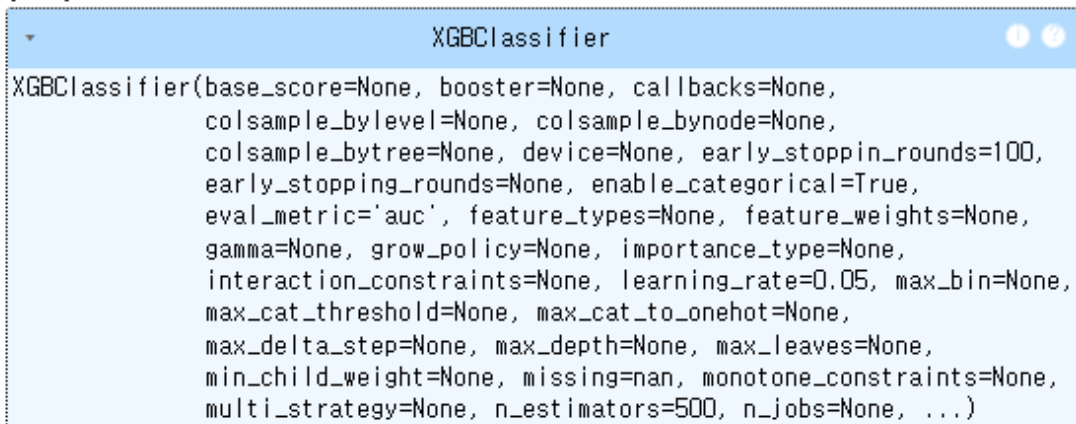
(1) XGB 학습 모델 생성

```
from xgboost import XGBClassifier
from sklearn.metrics import roc_auc_score

xgb_clf = XGBClassifier(n_estimators=500, learning_rate=0.05, random_state=156,
                        eval_metric='auc', early_stopping_rounds=100)

xgb_clf.fit(X_tr, y_tr, eval_set=[(X_tr, y_tr), (X_val, y_val)])
```

```
[494] validation_0-auc:0.93734 validation_1-auc:0.82836
[495] validation_0-auc:0.93738 validation_1-auc:0.82826
[496] validation_0-auc:0.93743 validation_1-auc:0.82817
[497] validation_0-auc:0.93746 validation_1-auc:0.82818
[498] validation_0-auc:0.93755 validation_1-auc:0.82815
[499] validation_0-auc:0.93767 validation_1-auc:0.82822
```



The screenshot shows a Jupyter Notebook cell with the title 'XGBClassifier'. The cell contains the following code: `XGBClassifier(base_score=None, booster=None, callbacks=None, colsample_bylevel=None, colsample_bynode=None, colsample_bytree=None, device=None, early_stopping_rounds=100, early_stopping_rounds=None, enable_categorical=True, eval_metric='auc', feature_types=None, feature_weights=None, gamma=None, grow_policy=None, importance_type=None, interaction_constraints=None, learning_rate=0.05, max_bin=None, max_cat_threshold=None, max_cat_to_onehot=None, max_delta_step=None, max_depth=None, max_leaves=None, min_child_weight=None, missing=nan, monotone_constraints=None, multi_strategy=None, n_estimators=500, n_jobs=None, ...)`

- `n_estimators` 는 500, `early_stopping_rounds` 는 100으로 설정하고 학습

(2) 성능 평가

```
xgb_roc_score = roc_auc_score(y_test, xgb_clf.predict_proba(X_test)[:, 1])
print('ROC AUC: {:.4f}'.format(xgb_roc_score))
```

ROC AUC: 0.8393

- ROC-AUC로 성능 평가
- 약 0.8393

(3) 하이퍼 파라미터 튜닝

(3)-1. 하이퍼 파라미터 검색 공간 설정

```
from hyperopt import hp

xgb_search_space = {'max_depth':hp.quniform('max_depth', 5, 15, 1),
                    'min_child_weight':hp.quniform('min_child_weight', 1, 6, 1),
                    'colsample_bytree':hp.uniform('colsample_bytree', 0.5, 0.95),
                    'learning_rate':hp.uniform('learning_rate', 0.01, 0.2)}
```

(3)-2. 목적함수 설정

```
from sklearn.model_selection import KFold
from sklearn.metrics import roc_auc_score

def objective_func(search_space):
    xgb_clf = XGBClassifier(n_estimators=100, early_stopping_rounds=30, eval_metric="auc",
                           max_depth=int(search_space['max_depth']),
                           min_child_weight=int(search_space['min_child_weight']),
                           colsample_bytree=search_space['colsample_bytree']
                           )

    roc_auc_list = []
    kf = KFold(n_splits=3)

    # K-Fold로 나눈 학습/검증 인덱스를 이용해 각 fold의 학습 데이터와 검증 데이터를 생성
    for tr_index, val_index in kf.split(X_train):
        X_tr, y_tr = X_train.iloc[tr_index], y_train.iloc[tr_index]
        X_val, y_val = X_train.iloc[val_index], y_train.iloc[val_index]

        # 학습 수행
        xgb_clf.fit(X_tr, y_tr, eval_set=[(X_tr, y_tr), (X_val, y_val)])

        # 1로 예측한 확률값 추출 후 roc auc 계산하고 평균을 list에 담기
        score = roc_auc_score(y_val, xgb_clf.predict_proba(X_val)[:, 1])
        roc_auc_list.append(score)

    return -1 * np.mean(roc_auc_list)
```

- 3 Fold 교차 검증을 이용해 평균 ROC-AUC 값을 반환하되 -1을 곱해 최대 ROC-AUC 값이 최소 반환값이 되게 함
- estimators =100, early_stopping_rounds =30

(3)-3. 최적의 하이퍼 파라미터 도출

```

from hyperopt import fmin, tpe, Trials

trials = Trials()

# 최적 입력값 추출
best = fmin(fn=objective_func, space=xgb_search_space,
            algo=tpe.suggest, max_evals=50,
            trials=trials, rstate=np.random.default_rng(seed=30))
print('best:', best)

```

```

100%|#####| 50/50 [15:18<00:00, 18.37s/trial, best loss: -0.836303420114652]
best: {'colsample_bytree': np.float64(0.6614864341301582), 'learning_rate': np.float64(0.14750531020054664),

```

```

        'max_depth': np.float64(5.0), 'min_child_weight': np.float64(5.0)}

```

- max_eval = 50로 실행
- **결과:** colsample_bytree ≈ 0.6615,
learning_rate ≈ 0.1475, max_depth = 5.0, min_child_weight = 5.0

(4) 재학습

```

xgb_clf = XGBClassifier(n_estimators=500, learning_rate=round(best['learning_rate'], 5),
                        max_depth=int(best['max_depth']),
                        min_child_weight=int(best['min_child_weight']),
                        colsample_bytree=round(best['colsample_bytree'], 5),
                        early_stopping_rounds=100, eval_metric='auc'
                        )

```

```

xgb_clf.fit(X_tr, y_tr, eval_set=[(X_tr, y_tr), (X_val, y_val)])

```

```

xgb_roc_score = roc_auc_score(y_test, xgb_clf.predict_proba(X_test)[:,:1])
print(f'ROC AUC: {xgb_roc_score:.4f}')

```

	validation_0-auc:0.83393	validation_1-auc:0.83393
[126]	validation_0-auc:0.90335	validation_1-auc:0.83402
[127]	validation_0-auc:0.90354	validation_1-auc:0.83410
[128]	validation_0-auc:0.90368	validation_1-auc:0.83408
[129]	validation_0-auc:0.90372	validation_1-auc:0.83398
[130]	validation_0-auc:0.90396	validation_1-auc:0.83404
[131]	validation_0-auc:0.90414	validation_1-auc:0.83401
	ROC AUC: 0.8434	

- n_estimators = 500으로 증가
- 약 0.8393 -> 0.8434로 성능 개선

(5) 피쳐 중요도

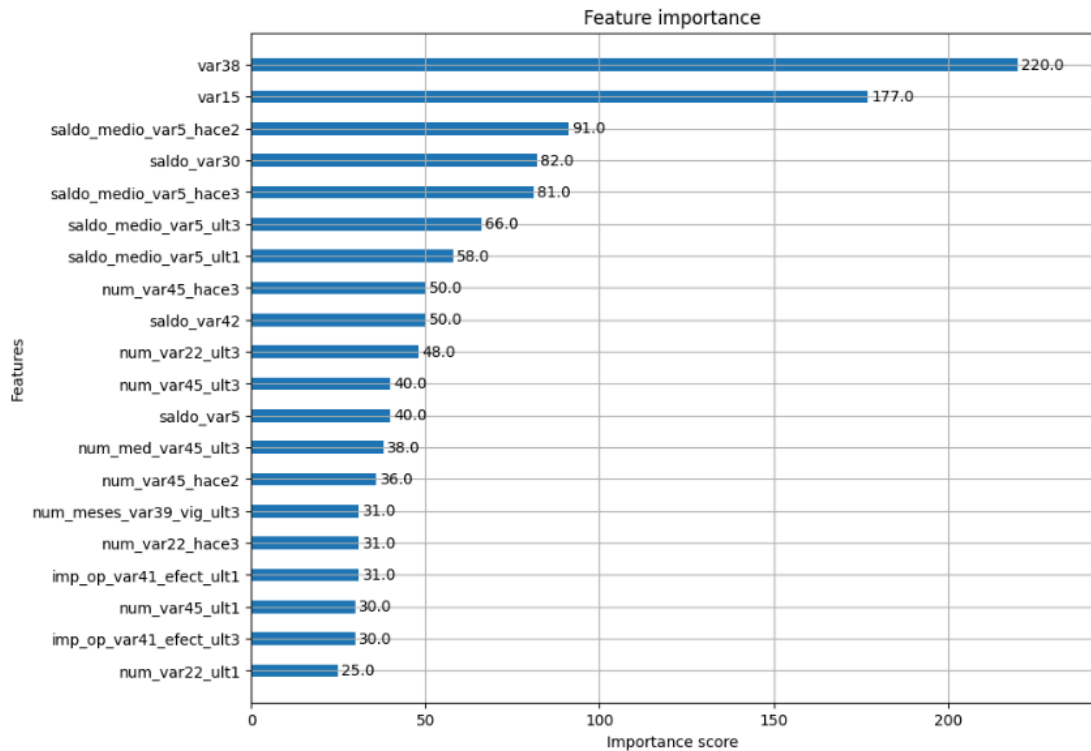
```

from xgboost import plot_importance
import matplotlib.pyplot as plt
%matplotlib inline

fig, ax = plt.subplots(1, 1, figsize=(10, 8))
plot_importance(xgb_clf, ax=ax, max_num_features=20, height=0.4)

<Axes: title={'center': 'Feature importance'}, xlabel='Importance score', ylabel='Features'>

```



- 가장 중요한 피쳐: 'var38'

3. LightGBM 모델 학습과 하이퍼 파라미터 튜닝

(1) LightGBM으로 학습 수행 및 성능 측정

```

from lightgbm import LGBMClassifier, early_stopping

lgbm_clf = LGBMClassifier(n_estimators=500)

eval_set = [(X_tr, y_tr), (X_val, y_val)]
lgbm_clf.fit(X_tr, y_tr, eval_metric='auc', eval_set=eval_set,
            callbacks=[early_stopping(stopping_rounds=100)])

lgbm_roc_score = roc_auc_score(y_test, lgbm_clf.predict_proba(X_test)[:,:1])
print('ROC AUC: {:.4f}'.format(lgbm_roc_score))

```

[LightGBM] [Info] Number of positive: 1658, number of negative: 40913
 [LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.125937 seconds.
 You can set 'force_row_wise=true' to remove the overhead.
 And if memory is not enough, you can set 'force_col_wise=true'.
 [LightGBM] [Info] Total Bins 13308
 [LightGBM] [Info] Number of data points in the train set: 42571, number of used features: 242
 [LightGBM] [Info] [binary:BoostFromScore]: pavg=0.038947 -> initscore=-3.205836
 [LightGBM] [Info] Start training from score -3.205836
 Training until validation scores don't improve for 100 rounds
 Early stopping, best iteration is:
 [42] training's auc: 0.91059 training's binary_logloss: 0.112183 valid_1's auc: 0.831787 valid_1's binary_logloss: 0.13527
 ROC AUC: 0.8384

- 약 0.8384

- 학습 시간 단축

(2) HyperOpt를 이용하여 다양한 하이퍼 파라미터 튜닝

(2)-1. 검색 공간 설정

```
lgbm_search_space = {'num_leaves':hp.quniform('num_leaves', 32, 64, 1),
                     'max_depth':hp.quniform('max_depth', 100, 160, 1),
                     'min_child_samples':hp.quniform('min_child_samples', 60, 100, 1),
                     'subsample':hp.uniform('subsample', 0.7, 1),
                     'learning_rate':hp.uniform('learning_rate', 0.01, 0.2)
                     }
```

(2)-2. 목적 함수 설정

```
def objective_func(search_space):
    lgbm_clf = LGBMClassifier(n_estimators=100,
                              num_leaves=int(search_space['num_leaves']),
                              max_depth=int(search_space['max_depth']),
                              min_child_samples=int(search_space['min_child_samples']),
                              subsample=search_space['subsample'],
                              learning_rate=search_space['learning_rate'])

    roc_auc_list = []

    kf = KFold(n_splits=3)

    # X_train을 다시 학습과 검증용 데이터로 분리
    for tr_index, val_index in kf.split(X_train):
        X_tr, y_tr = X_train.iloc[tr_index], y_train.iloc[tr_index]
        X_val, y_val = X_train.iloc[val_index], y_train.iloc[val_index]

        lgbm_clf.fit(X_tr, y_tr, eval_metric='auc', eval_set=[(X_tr, y_tr), (X_val, y_val)],
                     callbacks=[early_stopping(stopping_rounds=30)])

        # 1로 예측한 확률값 추출 후 roc auc 계산하고 list에 결과값 담기
        score = roc_auc_score(y_val, lgbm_clf.predict_proba(X_val)[: , 1])
        roc_auc_list.append(score)

    return -1*np.mean(roc_auc_list)
```

(2)-3. 최적 하이퍼 파라미터 도출

```
from hyperopt import fmin, tpe, Trials

trials = Trials()

best = fmin(fn=objective_func, space=lgbm_search_space, algo=tpe.suggest,
           max_evals=50, trials=trials, rstate=np.random.default_rng(seed=30))

print('best:', best)
```

```
[75] training's auc: 0.906532 training's binary_logloss: 0.114833 valid_1's auc: 0.832376 valid_1's binary_logloss: 0.1349
100%|██████████| 50/50 [02:22<00:00, 2.85s/trial, best loss: -0.8343025461736355]
best: {'learning_rate': np.float64(0.04874305670512305), 'max_depth': np.float64(109.0), 'min_child_samples': np.float64(60.0),
```

```
'num_leaves': np.float64(36.0), 'subsample': np.float64(0.976039035165039)}
```

- max_eval = 50로 실행
- **결과**: learning_rate $\approx$ 0.04874, max_depth = 109.0, min_child_weight = 60.0, num_leaves = 36.0, subsample $\approx$ 0.9760

(3) 재학습

```
lgbm_clf = LGBMClassifier(n_estimators=500, num_leaves=int(best['num_leaves']),
                        max_depth=int(best['max_depth']),
                        min_child_samples=int(best['min_child_samples']),
                        subsample=round(best['subsample'], 5),
                        learning_rate=round(best['learning_rate'], 5))
```

```
lgbm_clf.fit(X_tr, y_tr, eval_metric='auc', eval_set=[(X_tr, y_tr), (X_val, y_val)],
            callbacks=[early_stopping(stopping_rounds=100)])
```

```
lgbm_roc_score = roc_auc_score(y_test, xgb_clf.predict_proba(X_test)[:,:1])
print('ROC AUC: {:.4f}'.format(lgbm_roc_score))
```

```
[LightGBM] [Info] Number of positive: 1658, number of negative: 40913
[LightGBM] [Info] Auto-choosing row-wise multi-threading, the overhead of testing was 0.067148 seconds.
You can set 'force_row_wise=true' to remove the overhead.
And if memory is not enough, you can set 'force_col_wise=true'.
[LightGBM] [Info] Total Bins 13005
[LightGBM] [Info] Number of data points in the train set: 42571, number of used features: 203
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.038947 -> initscore=-3.205836
[LightGBM] [Info] Start training from score -3.205836
Training until validation scores don't improve for 100 rounds
Early stopping, best iteration is:
[76] training's auc: 0.906532 training's binary_logloss: 0.114833 valid_1's auc: 0.832376 valid_1's binary_logloss: 0.1349
ROC AUC: 0.8393
```

- 약 0.8393

캐글 노트북

1. Decision Tree And Random Forest Classifier Models

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.graph_objects as go
import plotly.express as px
```

1. Data Review

(1) 데이터 기본 정보 확인

```
data = pd.read_csv("drug200.csv")
```

data

	Age	Sex	BP	Cholesterol	Na_to_K	Drug
0	23	F	HIGH	HIGH	25.355	DrugY
1	47	M	LOW	HIGH	13.093	drugC
2	47	M	LOW	HIGH	10.114	drugC
3	28	F	NORMAL	HIGH	7.798	drugX
4	61	F	LOW	HIGH	18.043	DrugY
...	...	...	...	...	...	...
195	56	F	LOW	HIGH	11.567	drugC
196	16	M	LOW	HIGH	12.006	drugC
197	52	M	NORMAL	HIGH	9.894	drugX
198	23	M	NORMAL	NORMAL	14.020	drugX
199	40	F	LOW	NORMAL	11.349	drugX

200 rows × 6 columns

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 200 entries, 0 to 199  
Data columns (total 6 columns):  
#   Column      Non-Null Count  Dtype  
---  ---  
0   Age         200 non-null    int64  
1   Sex         200 non-null    object  
2   BP          200 non-null    object  
3   Cholesterol 200 non-null    object  
4   Na_to_K     200 non-null    float64  
5   Drug        200 non-null    object  
dtypes: float64(1), int64(1), object(4)  
memory usage: 9.5+ KB
```

```
data.describe()
```

	Age	Na_to_K
count	200.000000	200.000000
mean	44.315000	16.084485
std	16.544315	7.223956
min	15.000000	6.269000
25%	31.000000	10.445500
50%	45.000000	13.936500
75%	58.000000	19.380000
max	74.000000	38.247000

- 총 200개의 데이터와 6개의 칼럼으로 구성
- Null값은 존재X
- Age는 int형, Na_to_K는 float형, 나머지 4개 칼럼은 object형
- Age 평균은 약 44.3세, Na_to_K 평균은 약 16.08

(2) 데이터 관계 확인

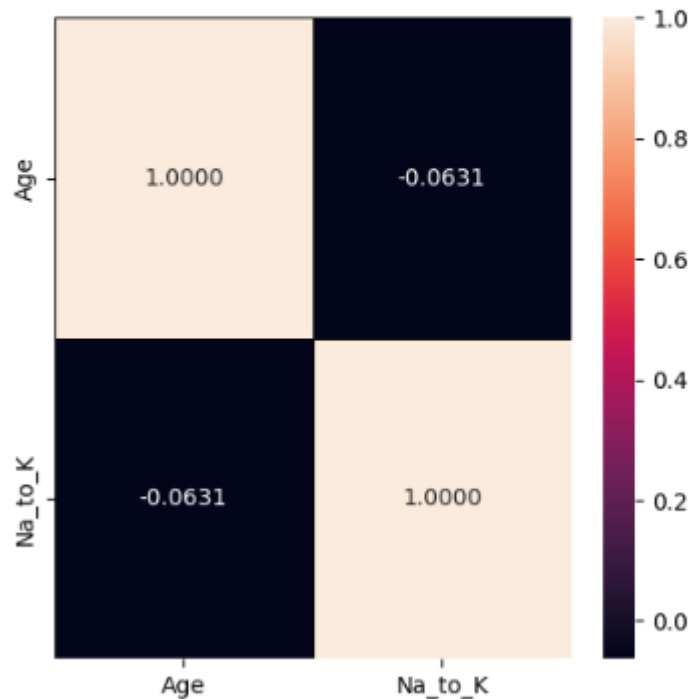
```
data.corr(numeric_only=True)
```

	Age	Na_to_K
Age	1.000000	-0.063119
Na_to_K	-0.063119	1.000000

- Age 와 Na_to_K 의 상관계수는 약 -0.063으로, 0에 매우 가까움
- 음수이므로 방향은 음의 상관관계이지만, 그 정도가 매우 약함

(2)-2. 시각화

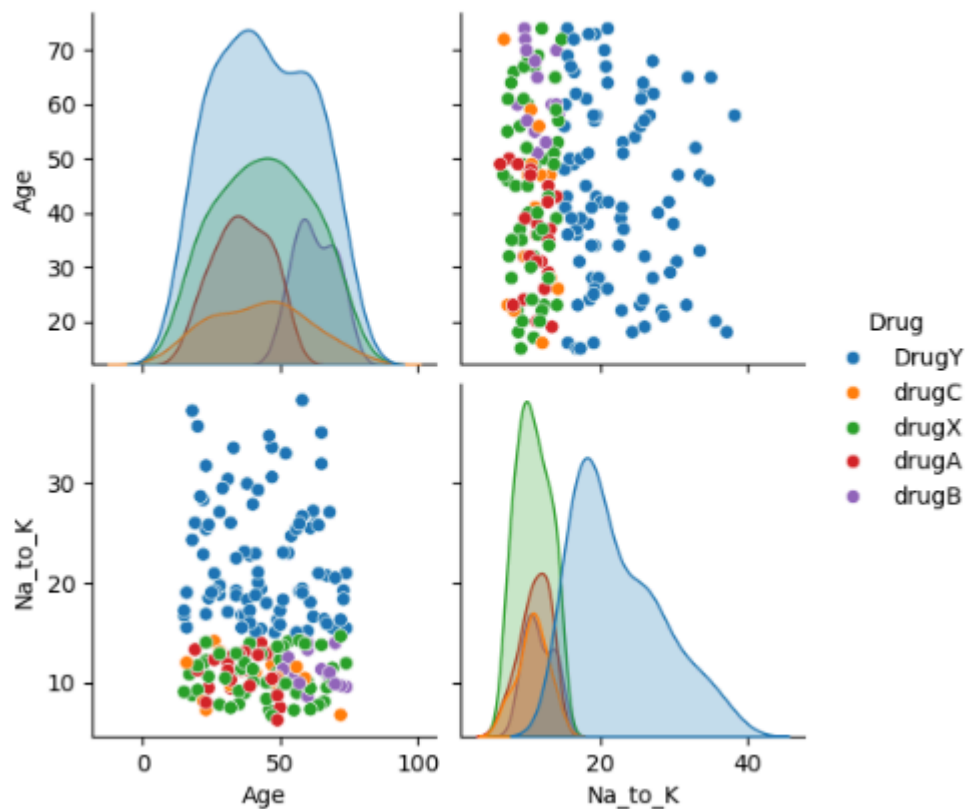
```
f, ax = plt.subplots(figsize=(5, 5))
sns.heatmap(data.corr(numeric_only=True), annot=True, linewidths=0.5,
            linecolor='black', fmt='.4f', ax=ax)
plt.show()
```



- Age 와 Na_to_K 의 상관계수 시각화

```
sns.pairplot(data, hue = "Drug")
```

<seaborn.axisgrid.PairGrid at 0x7ea1d2befb60>



- 수치형 변수 간 관계와 분포를 Drug 종류별로 구분하여 시각화

(3) 각 컬럼 안에 어떤 값들이 들어있는지 확인하기

```
data.columns
```

```
Index(['Age', 'Sex', 'BP', 'Cholesterol', 'Na_to_K', 'Drug'], dtype='object')
```

```
data['Age'].value_counts(dropna=False)
```

count	
Age	
47	8
23	7
28	7
49	7
32	6
39	6
50	5
60	5
...	...

```
data['Sex'].value_counts()
```

count	
Sex	
M	104
F	96

```
dtype: int64
```

```
data['BP'].value_counts()
```

count	
BP	
HIGH	77
LOW	64
NORMAL	59

```
dtype: int64
```

```
data['Cholesterol'].value_counts()
```

count	
Cholesterol	
HIGH	103
NORMAL	97

dtype: int64

```
data['Drug'].value_counts()
```

count	
Drug	
DrugY	91
drugX	54
drugA	23
drugC	16
drugB	16

dtype: int64

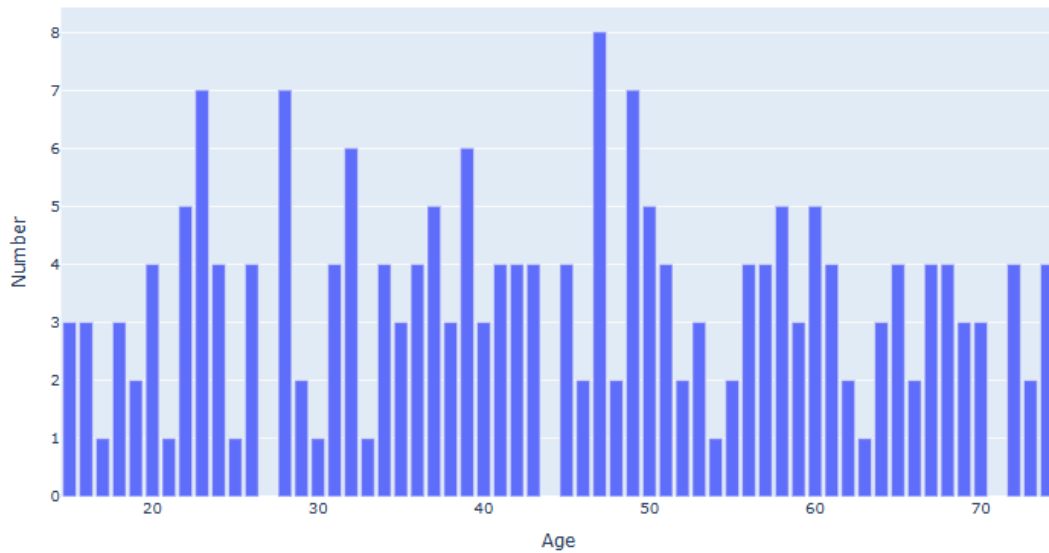
2. Data Visualization

(1) 연령별 환자 수를 시각화

```
dataAge = data['Age'].value_counts(dropna=False)
npar_dataAge = np.array(dataAge)
x = list(npar_dataAge)
y = dataAge.value_counts().index

DataAge = {'Age': y, 'Number': x}
DataAge = pd.DataFrame(DataAge)

fig = px.bar(DataAge, x = 'Age', y = 'Number')
fig.show()
```



- 환자들의 연령은 대략 15~74세에 걸쳐 분포
- 특정 연령대에만 데이터가 심하게 몰려 있지는 않음
- 가장 많이 등장한 특정 나이는 8명 정도로 나타남

(2) 환자 200명의 성별 비율 시각화

```

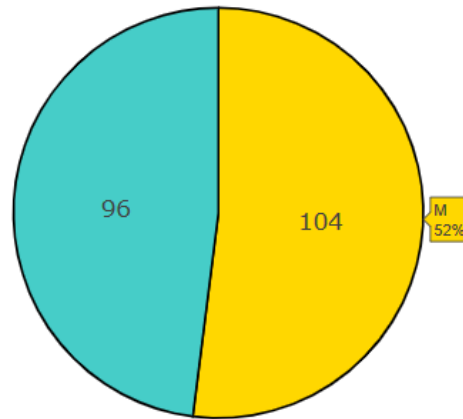
colors = ['gold', 'mediumturquoise']

fig = go.Figure(data = [go.Pie(labels= ['M', 'F'], values=[104, 96])])

fig.update_traces(hoverinfo = 'label + percent', textinfo = 'value', textfont_size=20,
                  marker = dict(colors = colors, line = dict(color = '#000000', width=2)))

fig.show()

```

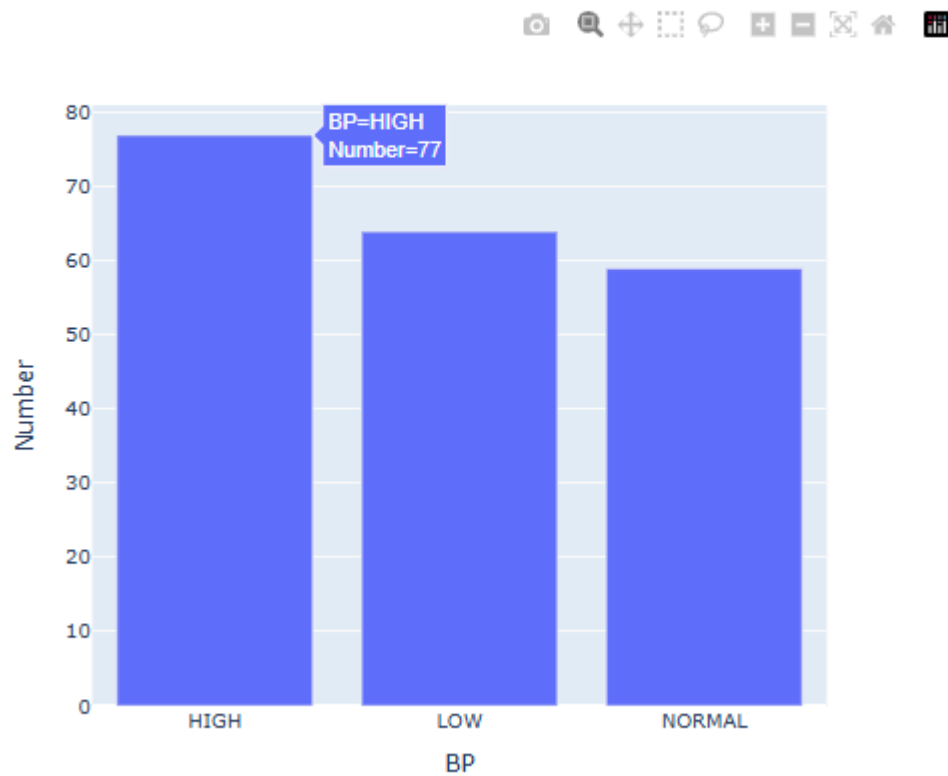


- 남성 104명(52%), 여성 96명(48%)
- 성별 분포가 비교적 균등

(3) 혈압(BP) 수준별 환자 수 시각화

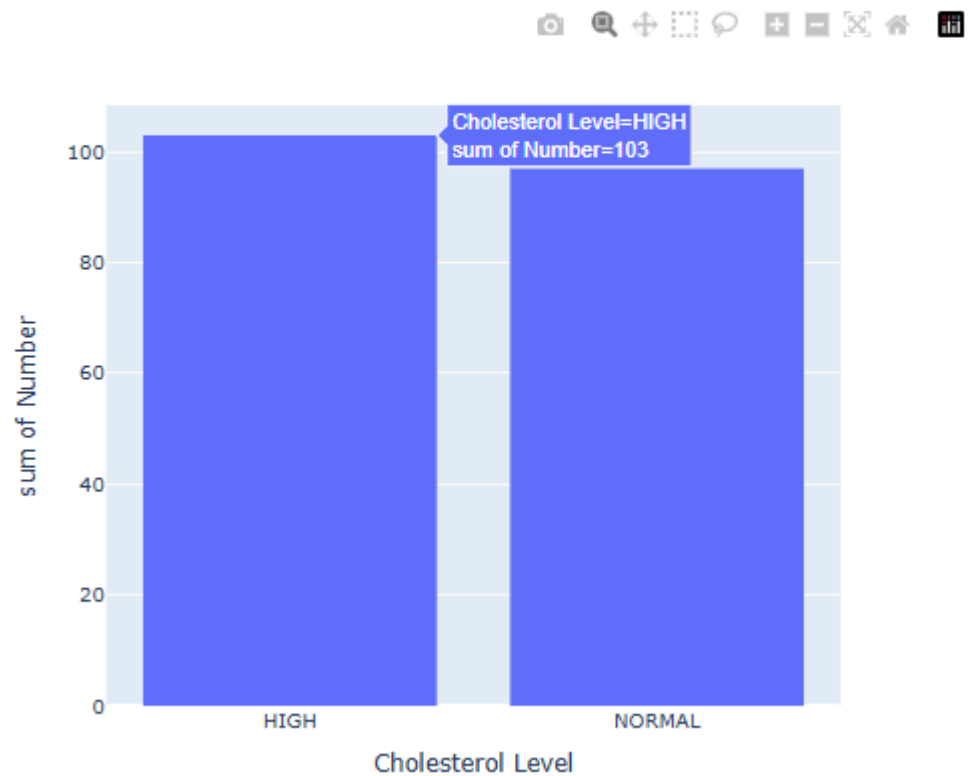
```
fig = px.bar(x = ['HIGH', 'LOW', 'NORMAL'], y = [77, 64, 59],  
            labels={'x': 'BP', 'y': 'Number'},  
            width=600, height=500, )
```

```
fig.show()
```



(3) 콜레스테롤(Cholesterol) 수준별 환자 수 시각화

```
fig = px.histogram(x = ['HIGH', 'NORMAL'], y = [103, 97],  
                  labels={'x':'Cholesterol Level', 'y':'Number'},  
                  width=600, height=500)  
  
fig.show()
```

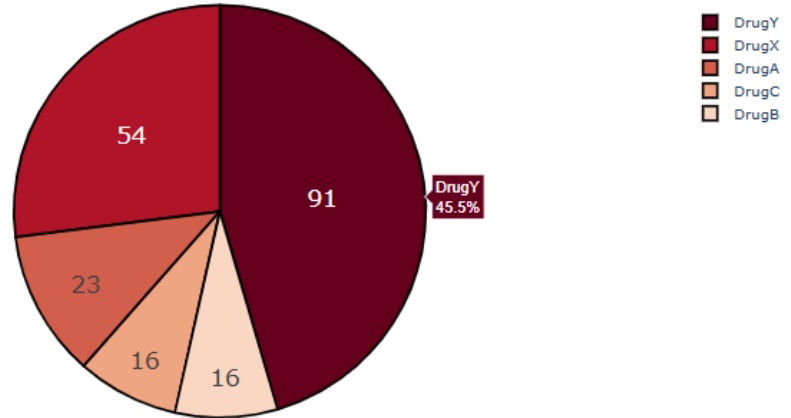


(4) 약물(Drug) 종류별 환자 수 시각화

```
fig = go.Figure(data = [go.Pie(labels=["DrugY", "DrugX", "DrugA", "DrugC", "DrugB"],
                                values=[91,54,23,16,16])])

fig.update_traces(hoverinfo = 'label + percent', textinfo = 'value',
                  textfont_size = 20, marker = dict(colors = px.colors.sequential.RdBu,
                                                    line = dict( color = '#000000', width = 2)))

fig.show()
```



(5) Na_to_K, 나이(Age), 약물 종류(Drug)의 관계 시각화

```
fig = px.scatter(data, x = "Na_to_K", y="Age", color="Drug",  
                 size='Age', hover_data=['Na_to_K'], width=600)  
fig.show()
```



3. Data Preparing

(1) 데이터 살펴보기

```
dataclass = pd.read_csv('drug200.csv')
```

```
dataclass
```

	Age	Sex	BP	Cholesterol	Na_to_K	Drug
0	23	F	HIGH	HIGH	25.355	DrugY
1	47	M	LOW	HIGH	13.093	drugC
2	47	M	LOW	HIGH	10.114	drugC
3	28	F	NORMAL	HIGH	7.798	drugX
4	61	F	LOW	HIGH	18.043	DrugY
...	...	...	...	...	...	...
195	56	F	LOW	HIGH	11.567	drugC
196	16	M	LOW	HIGH	12.006	drugC
197	52	M	NORMAL	HIGH	9.894	drugX
198	23	M	NORMAL	NORMAL	14.020	drugX
199	40	F	LOW	NORMAL	11.349	drugX

200 rows × 6 columns

(2) 인코딩

(2)-1. 성별 타입 변경

```
dataclass.Sex = [1 if i == 'F' else 0 for i in dataclass.Sex]
```

```
dataclass
```

	Age	Sex	BP	Cholesterol	Na_to_K	Drug
0	23	1	HIGH	HIGH	25.355	DrugY
1	47	0	LOW	HIGH	13.093	drugC
2	47	0	LOW	HIGH	10.114	drugC
3	28	1	NORMAL	HIGH	7.798	drugX
4	61	1	LOW	HIGH	18.043	DrugY
...	...	...	...	...	...	...
195	56	1	LOW	HIGH	11.567	drugC
196	16	0	LOW	HIGH	12.006	drugC
197	52	0	NORMAL	HIGH	9.894	drugX
198	23	0	NORMAL	NORMAL	14.020	drugX
199	40	1	LOW	NORMAL	11.349	drugX

200 rows × 6 columns

- F -> 1, M -> 0으로 변경

(2)-2. 혈압(BP) 타입 변경

```
import warnings
warnings.filterwarnings('ignore')

for i in range(0, len(dataclass.BP)):
    if dataclass.BP[i] == 'LOW':
        dataclass.BP[i] = 2
    elif dataclass.BP[i] == 'NORMAL':
        dataclass.BP[i] = 1
    else:
        dataclass.BP[i] = 0
```

dataclass

	Age	Sex	BP	Cholesterol	Na_to_K	Drug
0	23	1	0	HIGH	25.355	DrugY
1	47	0	2	HIGH	13.093	drugC
2	47	0	2	HIGH	10.114	drugC
3	28	1	1	HIGH	7.798	drugX
4	61	1	2	HIGH	18.043	DrugY
...	...	...	...	...	...	...
195	56	1	2	HIGH	11.567	drugC
196	16	0	2	HIGH	12.006	drugC
197	52	0	1	HIGH	9.894	drugX
198	23	0	1	NORMAL	14.020	drugX
199	40	1	2	NORMAL	11.349	drugX

200 rows × 6 columns

- LOW -> 2, NORMAL -> 1, HIGH -> 0으로 변경

(2)-3. 콜레스테롤(Cholesterol) 타입 변경

```
dataclass.Cholesterol = [1 if i == 'HIGH' else 0 for i in dataclass.Cholesterol]
```

dataclass

	Age	Sex	BP	Cholesterol	Na_to_K	Drug
0	23	1	0	1	25.355	DrugY
1	47	0	2	1	13.093	drugC
2	47	0	2	1	10.114	drugC
3	28	1	1	1	7.798	drugX
4	61	1	2	1	18.043	DrugY
...	...	...	...	...	...	...
195	56	1	2	1	11.567	drugC
196	16	0	2	1	12.006	drugC
197	52	0	1	1	9.894	drugX
198	23	0	1	0	14.020	drugX
199	40	1	2	0	11.349	drugX

200 rows × 6 columns

- HIGH -> 1, NORMAL -> 0으로 변경

(2)-4. 약물(Drug) 타입 변경

```
import warnings
warnings.filterwarnings('ignore')

for i in range(0, len(dataclass)):
    if dataclass.Drug[i] == 'DrugY':
        dataclass.Drug[i] = 4
    elif dataclass.Drug[i] == 'drugX':
        dataclass.Drug[i] = 3
    elif dataclass.Drug[i] == 'drugA':
        dataclass.Drug[i] = 2
    elif dataclass.Drug[i] == 'drugC':
        dataclass.Drug[i] = 1
    else:
        dataclass.Drug[i] = 0
```

dataclass

	Age	Sex	BP	Cholesterol	Na_to_K	Drug
0	23	1	0	1	25.355	4
1	47	0	2	1	13.093	1
2	47	0	2	1	10.114	1
3	28	1	1	1	7.798	3
4	61	1	2	1	18.043	4
...	...	...	...	...	...	...
195	56	1	2	1	11.567	1
196	16	0	2	1	12.006	1
197	52	0	1	1	9.894	3
198	23	0	1	0	14.020	3
199	40	1	2	0	11.349	3

200 rows × 6 columns

- DrugY -> 4, drugX -> 3, drugA -> 2, drugC -> 1, drugB -> 0으로 변경

(2)-5. 데이터 타입 변경 !]

```
data_types_dict = {'BP':int, 'Drug':int}

dataclass = dataclass.astype(data_types_dict)

dataclass.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   Age             200 non-null   int64
1   Sex             200 non-null   int64
2   BP              200 non-null   int64
3   Cholesterol     200 non-null   int64
4   Na_to_K         200 non-null   float64
5   Drug            200 non-null   int64
dtypes: float64(1), int64(5)
memory usage: 9.5 KB
```

- BP와 Drug를 int형으로 변경

(3) 학습/테스트용 데이터 분리

```
x_data = dataclass.drop(['Drug'], axis=1)
y_data = dataclass.Drug.values

from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(x_data, y_data,
                                                    test_size=0.3, random_state=1)
```

Decision Tree Classifier

(1) 결정 트리 학습 및 정확도 평가

```
from sklearn.tree import DecisionTreeClassifier
from sklearn import metrics

dtc = DecisionTreeClassifier()

dtc.fit(x_train, y_train)
predict = dtc.predict(x_test)

print('The accuracy of the Decision Tree is', metrics.accuracy_score(predict, y_test))
```

The accuracy of the Decision Tree is 0.9666666666666667

- 정확도 약 0.9667

(2) max_depth=3 일 때 gini 계수를 사용하여 정확도 평가

(2)-1.테스트 데이터 정확도 평가

```
from sklearn.metrics import accuracy_score

DTC_gini = DecisionTreeClassifier(criterion='gini',
                                  max_depth=3, random_state=0)

DTC_gini.fit(x_train, y_train)
y_pred_gini = DTC_gini.predict(x_test)

print('Model accuracy score with criterion gini index: {:.4f}'.format(
    accuracy_score(y_test, y_pred_gini)
))
```

Model accuracy score with criterion gini index: 0.9000

- 약 0.9000

(2)-2. 학습 데이터로 정확도 평가

```
y_pred_train_gini = DTC_gini.predict(x_train)
```

```
y_pred_train_gini
```

```
array([3, 3, 2, 4, 2, 3, 3, 3, 4, 2, 0, 3, 4, 3, 4, 3, 4, 4, 4, 4,
       3, 4, 2, 4, 3, 3, 2, 3, 4, 2, 0, 0, 3, 3, 3, 3, 3, 4, 4, 4, 4, 3,
       3, 0, 0, 2, 4, 3, 4, 3, 4, 4, 3, 3, 4, 4, 2, 4, 4, 4, 2, 3, 4, 4,
       4, 3, 4, 3, 3, 2, 3, 2, 4, 4, 4, 4, 3, 4, 0, 3, 3, 4, 0, 4, 0, 4,
       4, 0, 4, 4, 2, 3, 4, 3, 2, 4, 3, 4, 4, 3, 4, 4, 4, 4, 4, 3, 4,
       4, 2, 3, 3, 3, 4, 3, 4, 4, 4, 0, 4, 2, 3, 3, 2, 4, 4, 4, 4, 4, 3,
       2, 4, 3, 4, 2, 3, 2, 3])
```

```
print(y_train)
```

```
print(np.unique(y_train, return_counts=True))
```

```
[3 3 2 4 2 3 3 3 4 2 0 1 4 1 4 3 4 3 4 4 4 4 3 4 2 4 1 3 2 3 4 2 0 0 3 3 1
 3 3 4 4 4 4 3 1 0 0 2 4 3 4 1 4 4 1 3 4 4 2 4 4 4 2 3 4 4 4 3 4 1 3 2 3 2
 4 4 4 4 3 4 0 3 3 4 0 4 0 4 4 0 4 4 2 3 4 3 2 4 1 4 4 3 4 4 4 4 4 1 4 4
 2 3 1 1 4 3 4 4 4 0 4 2 3 3 2 4 4 4 4 3 2 4 3 4 2 3 2 3]
(array([0, 1, 2, 3, 4]), array([10, 12, 19, 35, 64]))
```

```
print('Training-set accuracy score: {:.4f}'.format(
    accuracy_score(y_train, y_pred_train_gini)
))
```

```
Training-set accuracy score: 0.9143
```

- 약 0.9143

(3) max_depth=3 일 때 엔트로피를 사용하여 정확도 평가

(3)-1.테스트 데이터 정확도 평가

```
DTC_en = DecisionTreeClassifier(criterion='entropy',
                                max_depth=3, random_state=0)
```

```
DTC_en.fit(x_train, y_train)
y_pred_en = DTC_en.predict(x_test)
```

```
print('Model accuracy score with criterion entropy: {:.4f}'.
      format(accuracy_score(y_test, y_pred_en)))
```

```
Model accuracy score with criterion entropy: 0.9000
```

- 약 0.9000

(3)-2. 학습 데이터로 정확도 평가

```
print('Training-set accuracy score: {:.4f}'.
      format(accuracy_score(y_train, y_pred_train_en)))
```

```
Training-set accuracy score: 0.9143
```

- 약 0.9143

Random Forest Classifier

(1) 랜덤포레스트 학습 및 정확도 평가 !

```
from sklearn.ensemble import RandomForestClassifier

rfc = RandomForestClassifier(random_state = 0)

rfc.fit(x_train, y_train)
predict = rfc.predict(x_test)

print('The accuracy of the RandomForest is',
      metrics.accuracy_score(predict, y_test))
```

The accuracy of the RandomForest is 0.95

- 정확도 0.95

(2) n_estimators=100 일 때 정확도 평가

```
from sklearn.ensemble import RandomForestClassifier

rfc_100 = RandomForestClassifier(n_estimators=100, random_state=0)

rfc_100.fit(x_train, y_train)
predict = rfc_100.predict(x_test)

print('The accuracy of the RandomForest is',
      metrics.accuracy_score(predict, y_test))
```

The accuracy of the RandomForest is 0.95

- 정확도 0.95

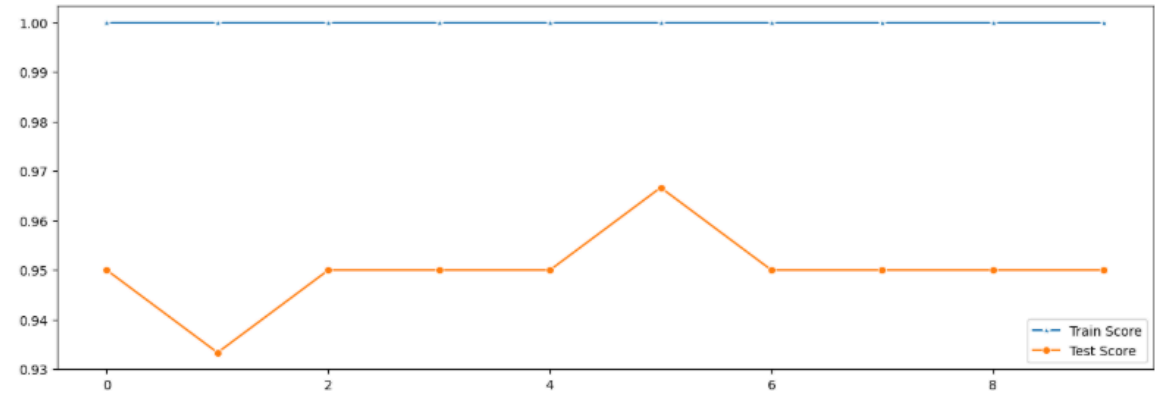
(3) 최적의 파라미터 찾기

(3)-1. random state값 찾기

```
test_score_list=[]
train_score_list=[]

for i in range(0, 10):
    rfc2 = RandomForestClassifier(random_state=i)
    rfc2.fit(x_train, y_train)
    test_score_list.append(rfc2.score(x_test, y_test))
    train_score_list.append(rfc2.score(x_train, y_train))

plt.figure(figsize=(15, 5))
p = sns.lineplot(x=range(0, 10), y=train_score_list, marker='*', label='Train Score')
p = sns.lineplot(x=range(0, 10), y=test_score_list, marker='o', label='Test Score')
```



- Random Forest는 모든 random state에서 학습 정확도 1.0을 기록
- 테스트 정확도는 약 0.933~0.967 사이로 나타남
- random_state=5 에서 테스트 정확도가 약 0.967로 가장 높음

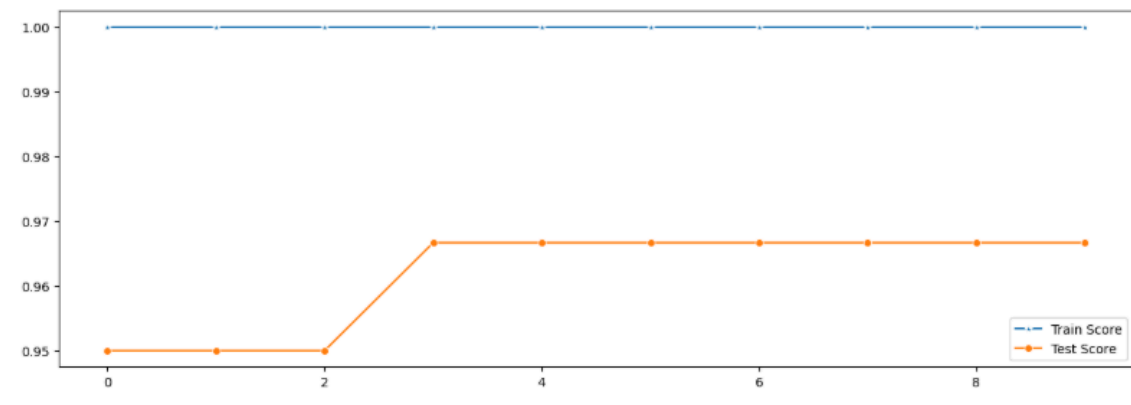
(3)-2. n_estimators값 찾기

```
test_score_list=[]
train_score_list=[]

list_n_estimators = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]

for i in range(0, len(list_n_estimators)):
    rfc3 = RandomForestClassifier(n_estimators=list_n_estimators[i],
                                random_state=5)
    rfc3.fit(x_train, y_train)
    test_score_list.append(rfc3.score(x_test, y_test))
    train_score_list.append(rfc3.score(x_train, y_train))

plt.figure(figsize=(15, 5))
p = sns.lineplot(x=range(0, len(list_n_estimators)), y=train_score_list,
                marker='*', label='Train Score')
p = sns.lineplot(x=range(0, len(list_n_estimators)), y=test_score_list,
                marker='o', label='Test Score')
```



- 트리 수가 40개일 때 테스트 정확도가 약 0.967로 증가
- 40 이후에는 동일한 성능을 유지

(4) 재학습

```
last_rfc = RandomForestClassifier(n_estimators=100, random_state=5)

last_rfc.fit(x_train, y_train)
predict = last_rfc.predict(x_test)

print('The accuracy of the Random Forest is',
      metrics.accuracy_score(predict, y_test))
```

The accuracy of the Random Forest is 0.9666666666666667

- 정확도 약 0.9667

Evaluation Classification Models

(1) 기본 설정 Decision Tree 모델의 혼동행렬

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix

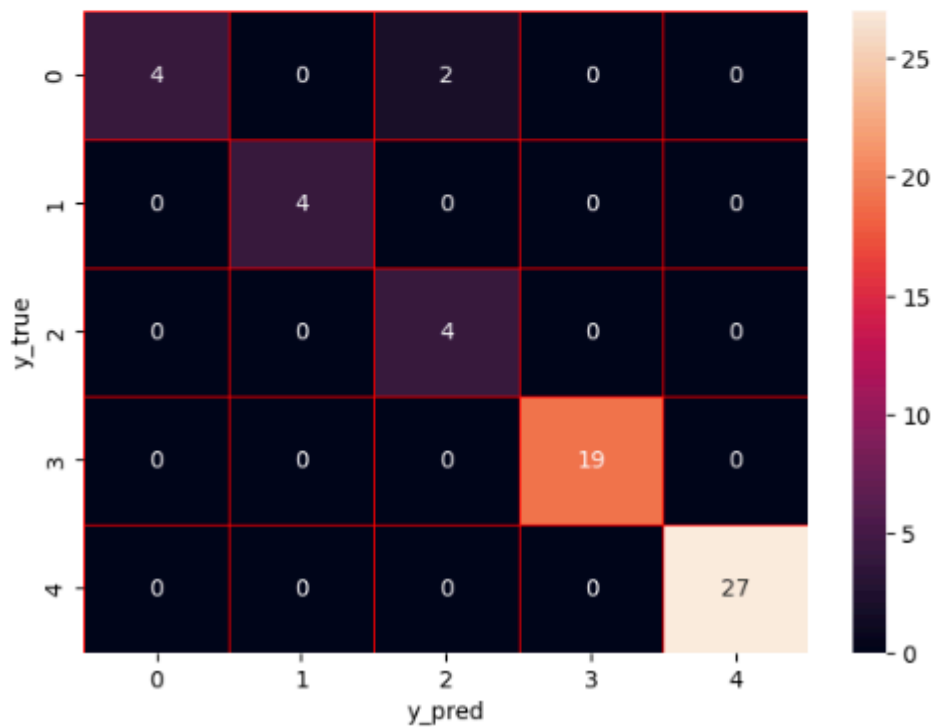
cm_des = DecisionTreeClassifier()

cm_des.fit(x_train, y_train)
y_pred_cm = cm_des.predict(x_test)
y_true = y_test

cm_des1 = confusion_matrix(y_true, y_pred_cm)

f, ax = plt.subplots(figsize = (7, 5))
sns.heatmap(cm_des1, annot=True, linewidths=0.5,
            linecolor='red', fmt='.0f', ax=ax)
plt.xlabel('y_pred')
plt.ylabel('y_true')
plt.show()

```



- 대부분의 데이터가 대각선에 위치하여 정확하게 분류
- 실제 클래스 0인 데이터 2개가 클래스 2로 오분류

(2) max_depth=3 일 때 gini 계수를 사용한 Decision Tree 모델의 혼동행렬

```

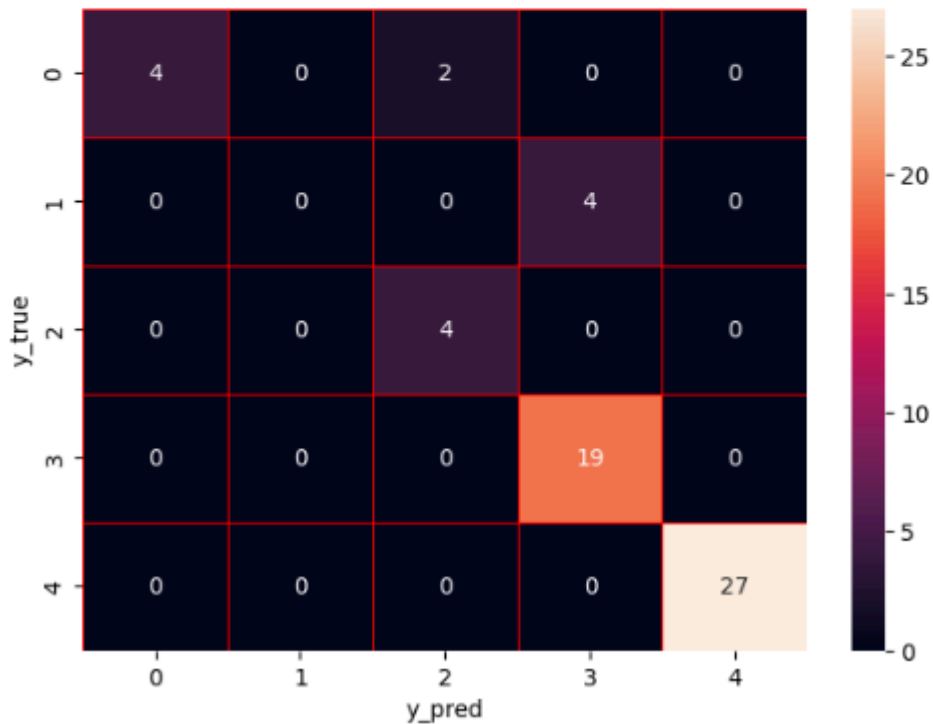
cm_des_gini = DecisionTreeClassifier(criterion='gini', max_depth=3, random_state=0)

cm_des_gini.fit(x_train,y_train)
y_pred_cm = cm_des_gini.predict(x_test)
y_true = y_test

cm_des2 = confusion_matrix( y_true, y_pred_cm)

f, ax = plt.subplots(figsize = (7,5))
sns.heatmap(cm_des2, annot = True, linewidths=0.5, linecolor="red", fmt = ".0f", ax = ax)
plt.xlabel("y_pred")
plt.ylabel("y_true")
plt.show()

```



- 실제 클래스 0의 2개와 클래스 1의 4개가 오분류

(3) 최적 파라미터로 학습한 Random Forest 모델의 혼동행렬

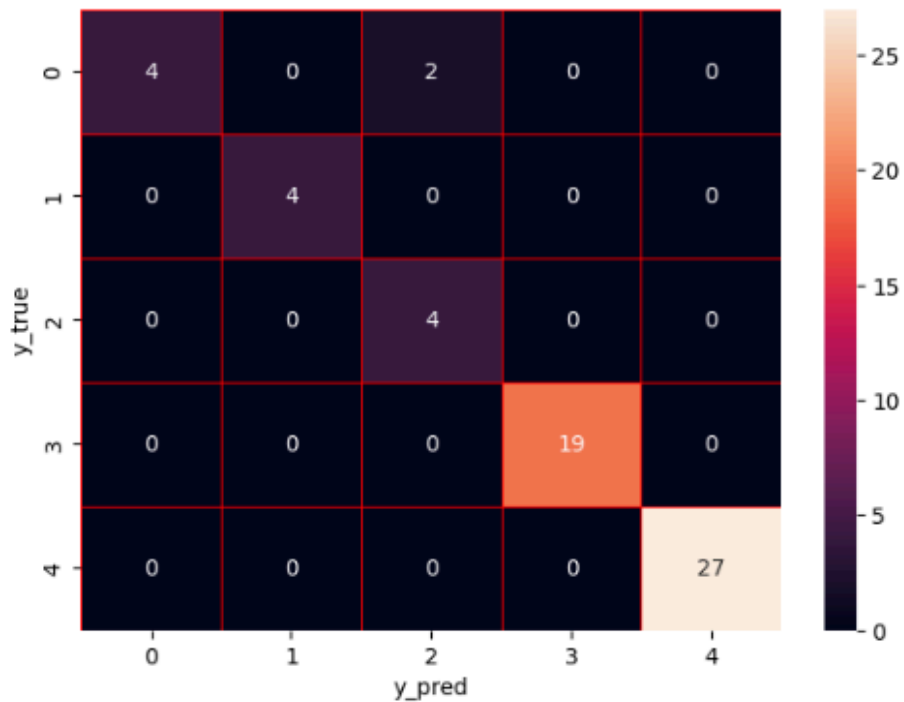
```
cm_last_rfc = RandomForestClassifier(n_estimators=100, random_state=5)

cm_last_rfc.fit(x_train, y_train)

y_pred_cm = cm_last_rfc.predict(x_test)
y_true = y_test

cm_rfc = confusion_matrix(y_true, y_pred_cm)

f, ax = plt.subplots(figsize = (7,5))
sns.heatmap(cm_rfc, annot = True, linewidths=0.5, linecolor="red", fmt = ".0f", ax = ax)
plt.xlabel("y_pred")
plt.ylabel("y_true")
plt.show()
```



- 실제 클래스 0인 데이터 2개가 클래스 2로 오분류

2. Beginner Friendly CATBOOST with OPTUNA

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import KFold, cross_val_score, RepeatedStratifiedKFold, StratifiedKFold, train_test_split
from sklearn.impute import SimpleImputer
from sklearn.pipeline import Pipeline
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.preprocessing import OneHotEncoder, StandardScaler, PowerTransformer
from sklearn.compose import ColumnTransformer, make_column_transformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.impute import SimpleImputer
from sklearn.dummy import DummyClassifier
from imblearn.over_sampling import SMOTE

```

1. Exploratory Data Analysis

(1) 데이터 정보 확인

(1)-1. 기본 정보 확인

```
df.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 918 entries, 0 to 917
Data columns (total 12 columns):
 #   Column              Non-Null Count  Dtype  
---  -
 0   Age                 918 non-null   int64  
 1   Sex                  918 non-null   object  
 2   ChestPainType        918 non-null   object  
 3   RestingBP            918 non-null   int64  
 4   Cholesterol           918 non-null   int64  
 5   FastingBS            918 non-null   int64  
 6   RestingECG           918 non-null   object  
 7   MaxHR                918 non-null   int64  
 8   ExerciseAngina        918 non-null   object  
 9   Oldpeak              918 non-null   float64  
10   ST_Slope             918 non-null   object  
11   HeartDisease          918 non-null   int64  
dtypes: float64(1), int64(6), object(5)
memory usage: 86.2+ KB

```

```
df.duplicated().sum()
```

```
np.int64(0)
```

- 총 918개의 데이터와 12개의 칼럼으로 구성
- Null값 존재X
- float형 1개, int형 6개, object형 5개
- 중복된 행 없음

(1)-2. 결측치 확인

```
def missing(df):
    missing_number = df.isnull().sum().sort_values(ascending=False)
    missing_percent = (df.isnull().sum()/df.isnull().count()).sort_values(ascending=False)
    missing_values = pd.concat([missing_number, missing_percent], axis=1,
                               keys=['Missing_Number', 'Missing_Percent'])

    return missing_values

missing(df)
```

	Missing_Number	Missing_Percent
Age	0	0.0
Sex	0	0.0
ChestPainType	0	0.0
RestingBP	0	0.0
Cholesterol	0	0.0
FastingBS	0	0.0
RestingECG	0	0.0
MaxHR	0	0.0
ExerciseAngina	0	0.0
Oldpeak	0	0.0
ST_Slope	0	0.0
HeartDisease	0	0.0

- 결측치 없음

(1)-3. 칼럼 타입 확인

```
numerical = df.drop(['HeartDisease'], axis=1).select_dtypes('number').columns
categorical = df.select_dtypes('object').columns

print(f'Numerical Columns: {df[numerical].columns}')
print('\n')
print(f'Categorical Columns: {df[categorical].columns}')
```

Numerical Columns: Index(['Age', 'RestingBP', 'Cholesterol', 'FastingBS', 'MaxHR', 'Oldpeak'], dtype='object')

Categorical Columns: Index(['Sex', 'ChestPainType', 'RestingECG', 'ExerciseAngina', 'ST_Slope'], dtype='object')

- Age, RestingBP, Cholesterol, FastingBS, MaxHR, Oldpeak 은 수치형 변수
- Sex, ChestPainType, RestingECG, ExerciseAngina, ST_Slope 은 범주형 변수

(1)-4. 범주형 변수의 고유값 개수 확인

```
df[categorical].nunique()
```

0

Sex	2
ChestPainType	4
RestingECG	3
ExerciseAngina	2
ST_Slope	3

dtype: int64

- Sex 와 ExerciseAngina 는 2개, ChestPainType 은 4개, RestingECG 와 ST_Slope 는 각각 3개의 범주로 구성
- 분산이 0인 변수도 없고, 분산이 지나치게 큰 변수도 없음

(2) Target Variable

(2)-1. 타겟 변수 HeartDisease의 클래스 분포 확인

```
y = df['HeartDisease']  
print(f'Percentage of patient had a HeartDisease: {round(y.value_counts(normalize=True)[1]*100,2)} % '  
f'--> ({y.value_counts()[1]} patient)\nPercentage of patient did not have a HeartDisease: '  
f'{round(y.value_counts(normalize=True)[0]*100,2)} % --> ({y.value_counts()[0]} patient)')
```

Percentage of patient had a HeartDisease: 55.34 % --> (508 patient)

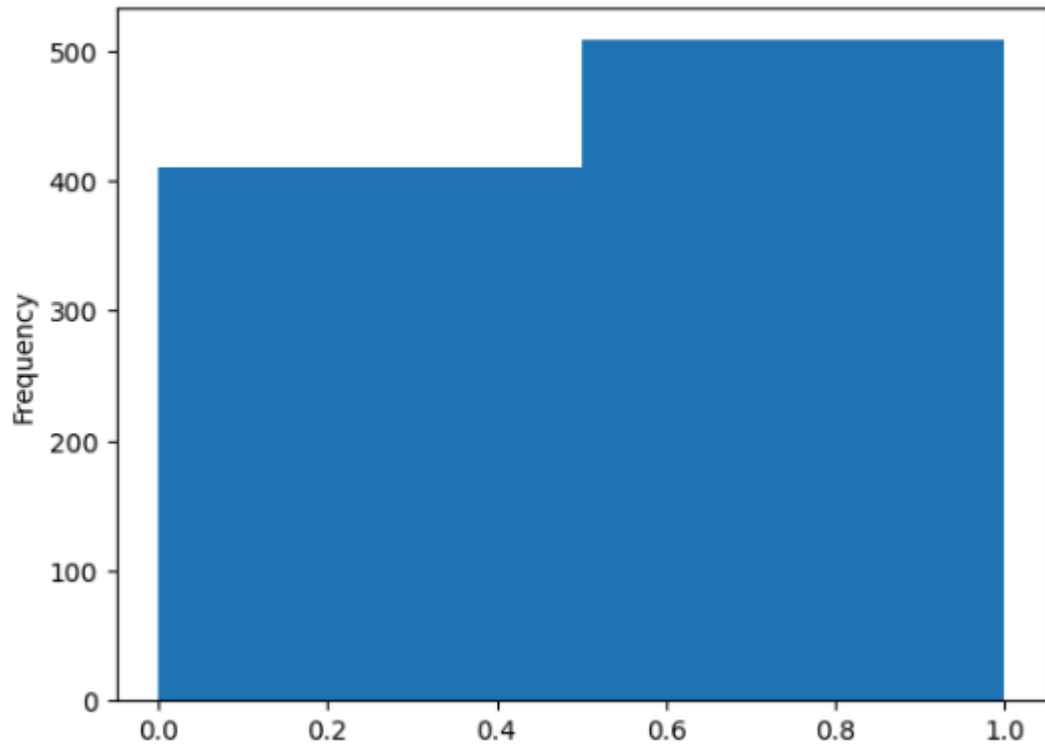
Percentage of patient did not have a HeartDisease: 44.66 % --> (410 patient)

- 전체 환자 중 508명(약 55.34%)이 심장질환을 가지고 있음
-전체 환자 중 410명(약 44.66%)이 심장질환을 가지고 있음

(2)-2. 시각화

```
df['HeartDisease'].plot(kind='hist', bins=[0, 0.5, 1])
```

<Axes: ylabel='Frequency'>



- 약간의 클래스 불균형은 존재하지만, 문제가 될 정도로 심하지 않음
- 모델의 평가 지표로 정확도(Accuracy) 사용 가능

(3) Numerical Features

(3)-1. 수치형 변수의 기초 통계량

```
df[numerical].describe()
```

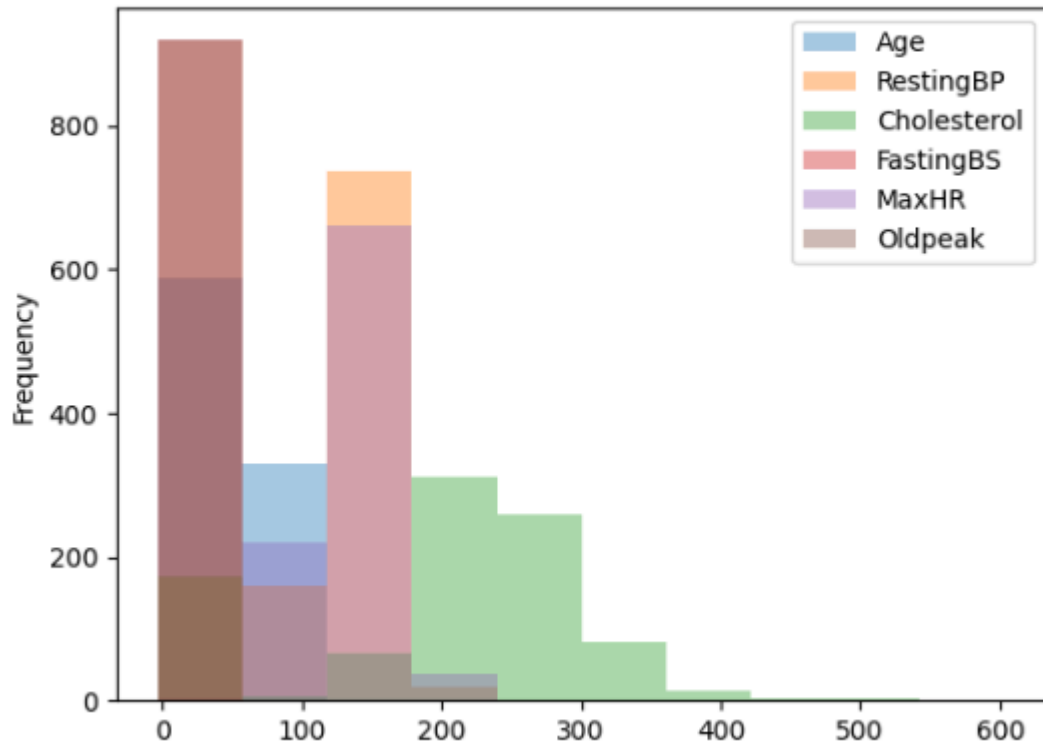
	Age	RestingBP	Cholesterol	FastingBS	MaxHR	Oldpeak
count	918.000000	918.000000	918.000000	918.000000	918.000000	918.000000
mean	53.510893	132.396514	198.799564	0.233115	136.809368	0.887364
std	9.432617	18.514154	109.384145	0.423046	25.460334	1.066570
min	28.000000	0.000000	0.000000	0.000000	60.000000	-2.600000
25%	47.000000	120.000000	173.250000	0.000000	120.000000	0.000000
50%	54.000000	130.000000	223.000000	0.000000	138.000000	0.600000
75%	60.000000	140.000000	267.000000	0.000000	156.000000	1.500000
max	77.000000	200.000000	603.000000	1.000000	202.000000	6.200000

- RestingBP 와 Cholesterol 에서 최솟값이 0으로 나타나 해당 값에 대한 추가 확인이 필요

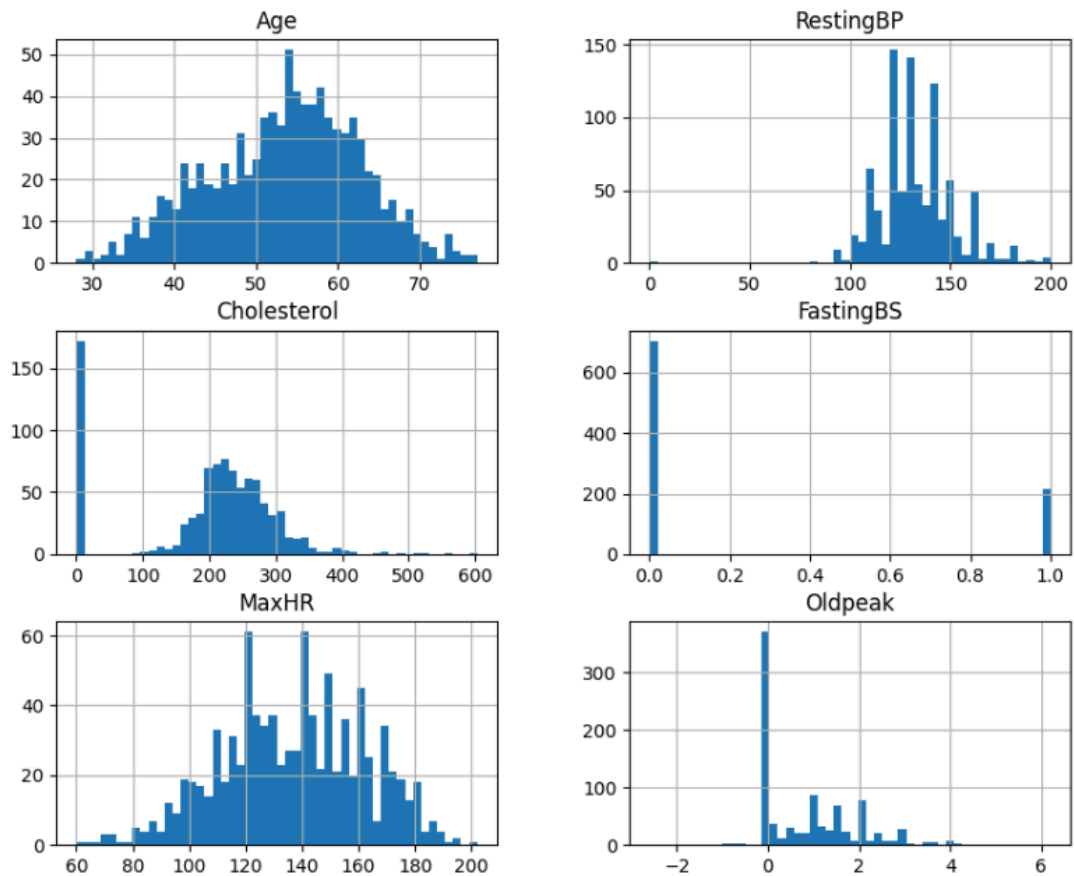
(3)-2. 시각화

```
df[numerical].plot(kind='hist', alpha=0.4)
```

<Axes: ylabel='Frequency'>



```
df[numerical].hist(bins=50, figsize=(10, 8));
```



(3)-3. 수치형 변수 중 왜도(skewness)가 큰 변수를 찾기

```
skew_limit = 0.75
skew_vals = df[numerical].drop('FastingBS', axis=1).skew()
skew_cols = skew_vals[abs(skew_vals)>skew_limit].sort_values(ascending=False)
skew_cols
```

0

Oldpeak 1.022872

dtype: float64

- Oldpeak의 왜도가 약 1.02로 나타나 분포가 한쪽으로 치우쳐 있음
- 왜도 측면에서 크게 문제되는 부분은 없음
- 수치형 변수들은 전반적으로 정규분포와 유사한 형태를 보임

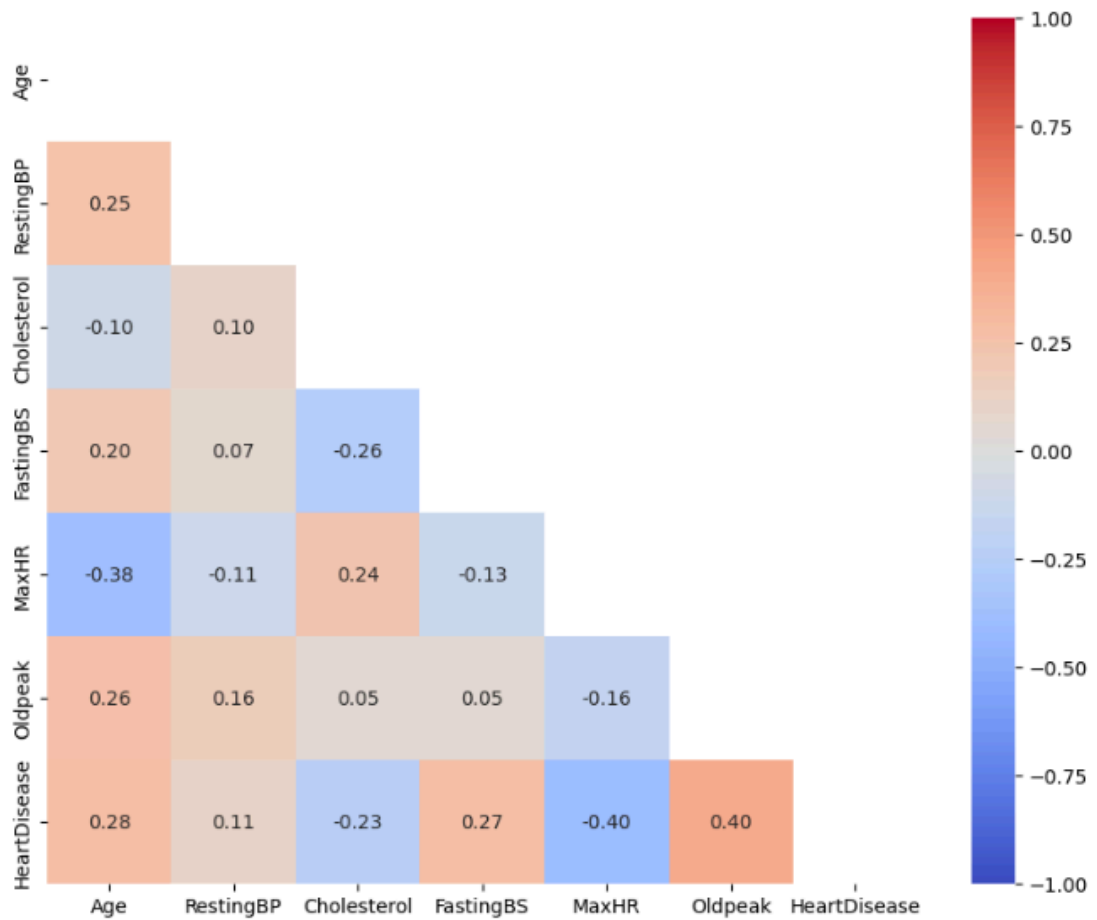
(3)-4. 변수끼리의 상관관계 시각화

```

numerical1 = df.select_dtypes('number').columns

matrix = np.triu(df[numerical1].corr())
fig, ax = plt.subplots(figsize=(10,8))
sns.heatmap(df[numerical1].corr(), annot=True, fmt='.2f', vmin=-1, vmax=1,
            center=0, cmap='coolwarm', mask=matrix, ax=ax);

```



- 수치형 변수들과 타겟 변수(HeartDisease) 사이에는 전반적으로 강하지 않은 상관관계가 나타남
- Oldpeak 은 심장질환 여부와 양의 상관관계(0.40)를 보였다.
- MaxHR (최대 심박수)은 심장질환 여부와 음의 상관관계(-0.40)를 보였다.
- Cholesterol 은 심장질환 여부와 음의 상관관계(-0.23)를 보였다.

(4) Categorical Features

(4)-1. 범주형 변수 확인하기

```
df[categorical].head()
```

	Sex	ChestPainType	RestingECG	ExerciseAngina	ST_Slope
0	M	ATA	Normal	N	Up
1	F	NAP	Normal	N	Flat
2	M	ATA	ST	N	Up
3	F	ASY	Normal	Y	Flat
4	M	NAP	Normal	N	Up

(4)-2. 성별에 따라 심장질환이 있는 비율

```
print (f'A female person has a probability of {round(df[df["Sex"]=="F"]["HeartDisease"].mean()*100,2)} % have a HeartDisease')
print()
print (f'A male person has a probability of {round(df[df["Sex"]=="M"]["HeartDisease"].mean()*100,2)} % have a HeartDisease')
```

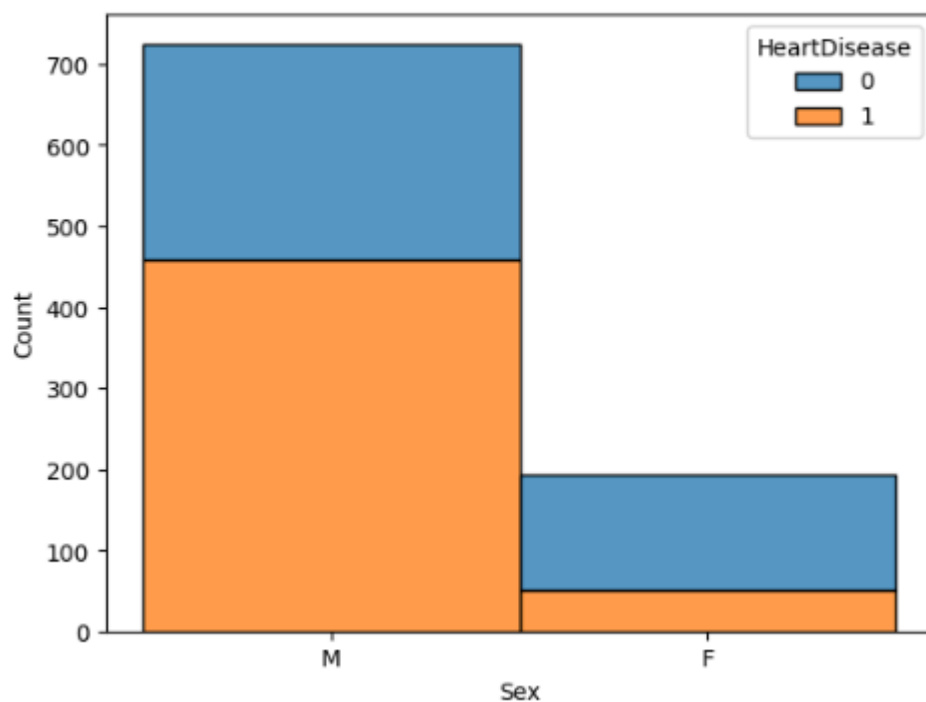
A female person has a probability of 25.91 % have a HeartDisease

A male person has a probability of 63.17 % have a HeartDisease

- 여성은 25.91%, 남성은 63.17%로 남성에서 심장질환 비율이 더 높게 나타남

```
sns.histplot(data=df, x='Sex', hue='HeartDisease', multiple='stack')
```

<Axes: xlabel='Sex', ylabel='Count'>



(4)-3. 흉통 유형(ChestPainType)별로 심장질환이 있는 환자의 비율

```
df.groupby('ChestPainType')['HeartDisease'].mean().sort_values(ascending=False)
```

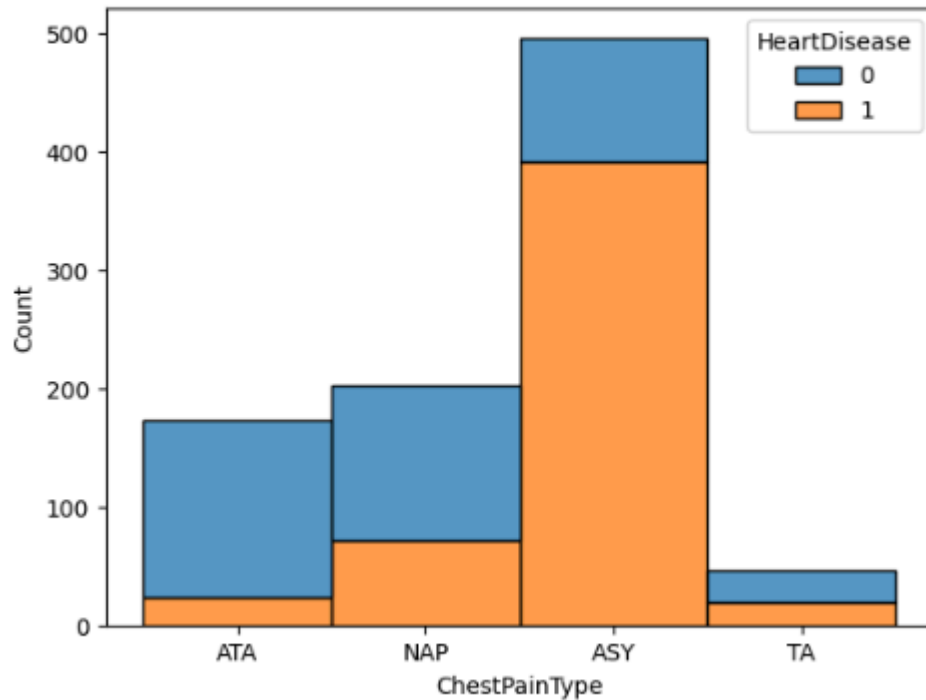
HeartDisease	
ChestPainType	
ASY	0.790323
TA	0.434783
NAP	0.354680
ATA	0.138728

dtype: float64

- ASY(무증상) 그룹이 약 79.03%로 가장 높음
- ATA(비전형적 협심증) 그룹이 약 13.87%로 가장 낮게 나타남

```
sns.histplot(data=df, x='ChestPainType', hue='HeartDisease', multiple='stack')
```

<Axes: xlabel='ChestPainType', ylabel='Count'>



- 흉통 유형에 따라 심장질환 비율에 뚜렷한 차이가 나타남
- ASY(무증상) 그룹은 ATA(비전형적 협심증) 그룹보다 심장질환이 있는 비율이 약 6배 높게 나타남

(4)-4. 안정 시 심전도(RestingECG) 유형별 심장질환 비율

```
df.groupby('RestingECG')['HeartDisease'].mean().sort_values(ascending=False)
```

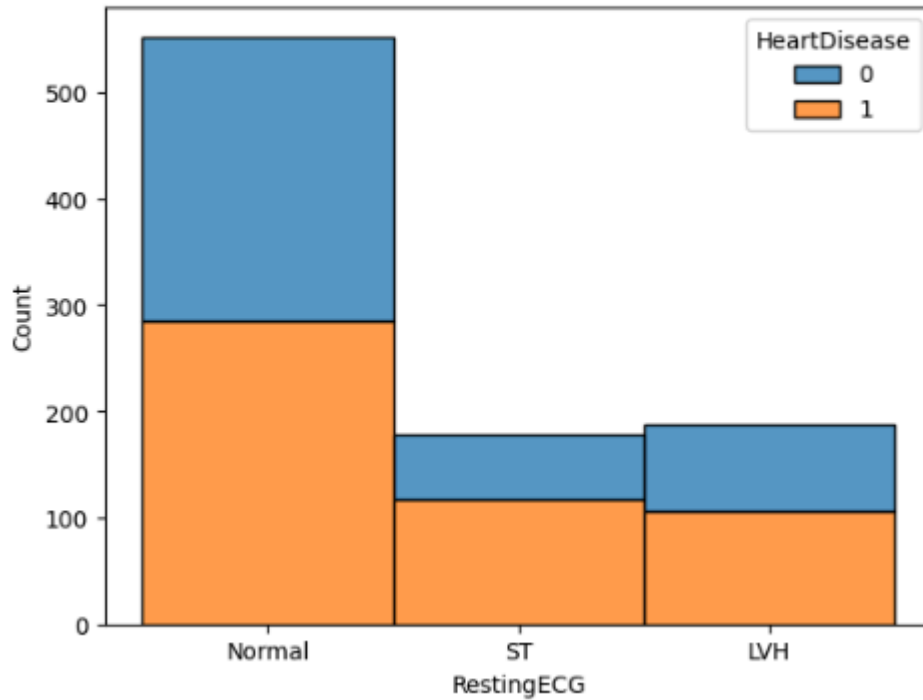
HeartDisease	
RestingECG	
ST	0.657303
LVH	0.563830
Normal	0.516304

dtype: float64

- ST 그룹이 약 65.73%로 가장 높았으며, LVH는 56.38%, Normal은 51.63%로 나타남

```
sns.histplot(data=df, x='RestingECG', hue='HeartDisease', multiple='stack')
```

<Axes: xlabel='RestingECG', ylabel='Count'>



- 안정 시 심전도(RestingECG) 유형에 따른 심장질환 비율의 차이는 크지 않음
- ST-T파 이상(ST)이 있는 그룹에서 다른 그룹보다 심장질환 비율이 상대적으로 높게 나타남

(4)-5. 운동 유발 협심증(ExerciseAngina) 여부에 따라 심장질환이 있는 비율

```
df.groupby('ExerciseAngina')['HeartDisease'].mean().sort_values(ascending=False)
```

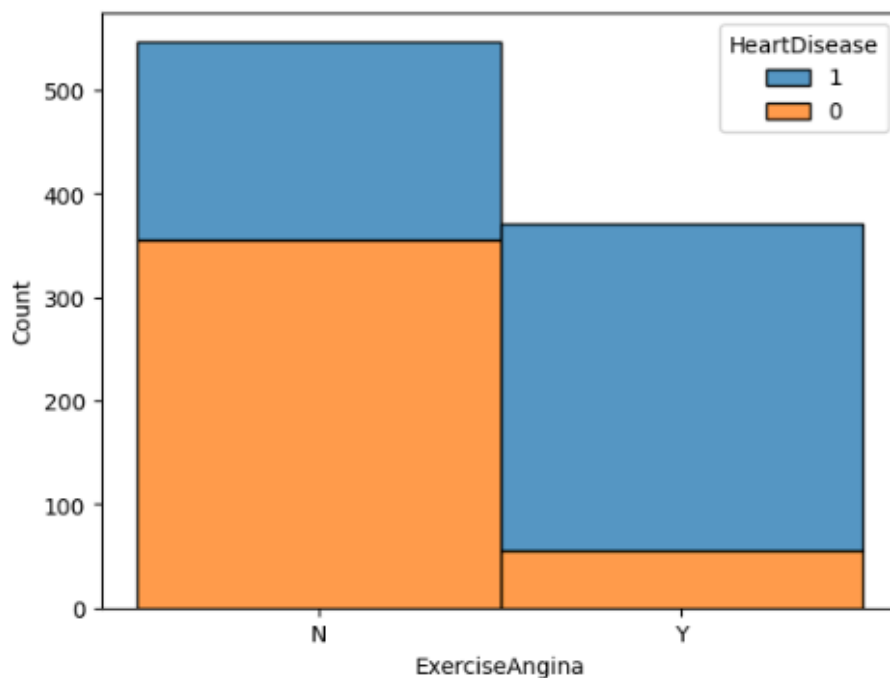
HeartDisease	
ExerciseAngina	
Y	0.851752
N	0.351005

dtype: float64

- 운동 유발 협심증이 있는 환자의 심장질환 비율은 약 85.18%로, 운동 유발 협심증이 없는 환자의 35.10%보다 높게 나타남

```
sns.histplot(data=df, x='ExerciseAngina', hue='HeartDisease', hue_order=[1, 0], multiple='stack')
```

<Axes: xlabel='ExerciseAngina', ylabel='Count'>



- 운동 유발 협심증이 있는(Y) 그룹은 없는(N) 그룹보다 심장질환 비율이 약 2.4배 높게 나타남

(4)-6. 운동 시 ST 구간의 기울기(ST_Slope) 유형별 심장질환 비율

```
df.groupby('ST_Slope')['HeartDisease'].mean().sort_values(ascending=False)
```

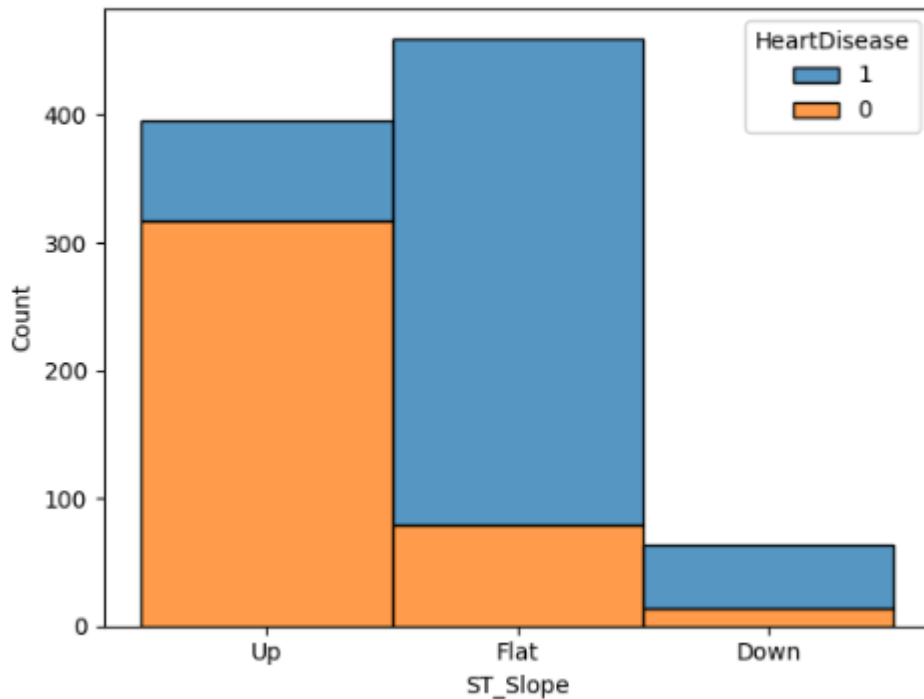
HeartDisease	
ST_Slope	
Flat	0.828261
Down	0.777778
Up	0.197468

dtype: float64

- Flat 과 Down 그룹은 각각 약 82.83%, 77.78%로 높게 나타난 반면, Up 그룹은 약 19.75%로 상대적으로 낮게 나타남

```
sns.histplot(data=df, x='ST_Slope', hue='HeartDisease', hue_order=[1, 0], multiple='stack')
```

<Axes: xlabel='ST_Slope', ylabel='Count'>



- 운동 시 ST 구간의 기울기(ST_Slope)에 따라 심장질환 비율에 뚜렷한 차이가 나타남
- Up 그룹은 Flat 과 Down 그룹에 비해 심장질환 비율이 현저히 낮게 나타남

(5) 인사이트 정리

- 타겟 변수인 심장질환 있음(55.34%)과 없음(44.66%)의 비율은 비교적 균형적으로 나타남
- 수치형 변수들은 전반적으로 HeartDisease와 강한 상관관계를 보이지 않음
- oldpeak 은 심장질환과 양의 상관관계(0.40), MaxHR 은 음의 상관관계(-0.40)를 보임
- Cholesterol 은 예상과 달리 심장질환과 약한 음의 상관관계(-0.23)를 보임
- 성별에 따른 심장질환 비율은 남성 63.17%, 여성 25.91%로, 남성이 여성보다 약 2.44배 높게 나타났다.
 흉통 유형에 따라 심장질환 비율에 뚜렷한 차이가 나타났으며, ASY(무증상)는 79.03%, ATA(비전형적 협심증)는 13.87%로 약 5.7배 차이가 나타남
- 안정 시 심전도(RestingECG) 유형별 차이는 상대적으로 크지 않았으나, ST 그룹의 심장질환 비율이 65.73%로 가장 높음

- 운동 유발 협심증(ExerciseAngina)이 있는 그룹은 심장질환 비율이 85.18%, 없는 그룹은 35.10%로 약 2.4배 차이가 나타남
- ST_Slope 에서도 뚜렷한 차이가 나타났으며, Flat 82.83%, Down 77.78%에 비해 Up 은 19.75%로 현저히 낮음

2. MODEL SELECTION

(1) Baseline Model

```
from sklearn.model_selection import train_test_split

accuracy = []
model_names = []

X= df.drop('HeartDisease', axis=1)
y= df['HeartDisease']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

ohe= OneHotEncoder()
ct= make_column_transformer((ohe,categorical),remainder='passthrough')

model = DummyClassifier(strategy='constant', constant=1)
pipe = make_pipeline(ct, model)
pipe.fit(X_train, y_train)
y_pred = pipe.predict(X_test)
accuracy.append(round(accuracy_score(y_test, y_pred),4))
print (f'model : {model} and accuracy score is : {round(accuracy_score(y_test, y_pred),4)}')

model_names = ['DummyClassifier']
dummy_result_df = pd.DataFrame({'Accuracy':accuracy}, index=model_names)
dummy_result_df
```

model : DummyClassifier(constant=1, strategy='constant') and accuracy score is : 0.5942

	Accuracy	
DummyClassifier	0.5942	

- 학습/테스트 데이터로 분리
- 범주형 변수 One-Hot Encoding
- HeartDisease=1 로 예측하는 DummyClassifier 생성
- 결과: 정확도 약 59.42%

(2) Logistic & Linear Discriminant & SVC & KNN

```
accuracy = []
model_names = []

X= df.drop('HeartDisease', axis=1)
y= df['HeartDisease']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

ohe= OneHotEncoder()
ct= make_column_transformer((ohe,categorical),remainder='passthrough')

lr = LogisticRegression(solver='liblinear')
lda= LinearDiscriminantAnalysis()
svm = SVC(gamma='scale')
knn = KNeighborsClassifier()

models = [lr,lda,svm,knn]

for model in models:
    pipe = make_pipeline(ct, model)
    pipe.fit(X_train, y_train)
    y_pred = pipe.predict(X_test)
    accuracy.append(round(accuracy_score(y_test, y_pred),4))
    print(f'model : {model} and accuracy score is : {round(accuracy_score(y_test, y_pred),4)}')

model_names = ['Logistic','LinearDiscriminant','SVM','KNeighbors']
result_df1 = pd.DataFrame({'Accuracy':accuracy}, index=model_names)
result_df1
```

```
model : LogisticRegression(solver='liblinear') and accuracy score is : 0.8841
model : LinearDiscriminantAnalysis() and accuracy score is : 0.8696
model : SVC() and accuracy score is : 0.7246
model : KNeighborsClassifier() and accuracy score is : 0.7174
```

Accuracy		
Logistic	0.8841	
LinearDiscriminant	0.8696	
SVM	0.7246	
KNeighbors	0.7174	

- 모든 모델이 DummyClassifier의 기준 정확도(59.42%)보다 높은 성능을 보임
- Logistic Regression은 정확도 약 88.41%
- LinearDiscriminantAnalysis은 정확도 약 86.96%
- SVC는 정확도 약 72.46%
- KNeighborsClassifier는 정확도 약 71.74%

(3) Logistic & Linear Discriminant & SVC & KNN with Scaler

```

accuracy = []
model_names = []

X= df.drop('HeartDisease', axis=1)
y= df['HeartDisease']

numerical = X.select_dtypes('number').columns
categorical = X.select_dtypes('object').columns
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

ohe= OneHotEncoder()
s= StandardScaler()
ctl= make_column_transformer((ohe,categorical),(s,numerical))

lr = LogisticRegression(solver='liblinear')
lda= LinearDiscriminantAnalysis()
svm = SVC(gamma='scale')
knn = KNeighborsClassifier()

models = [lr,lda,svm,knn]

for model in models:
    pipe = make_pipeline(ctl, model)
    pipe.fit(X_train, y_train)
    y_pred = pipe.predict(X_test)
    accuracy.append(round(accuracy_score(y_test, y_pred),4))
    print (f'model : {model} and accuracy score is : {round(accuracy_score(y_test, y_pred),4)}')

model_names = ['Logistic_scl', 'LinearDiscriminant_scl', 'SVM_scl', 'KNeighbors_scl']
result_df2 = pd.DataFrame({'Accuracy':accuracy}, index=model_names)
result_df2

```

```

model : LogisticRegression(solver='liblinear') and accuracy score is : 0.8804
model : LinearDiscriminantAnalysis() and accuracy score is : 0.8696
model : SVC() and accuracy score is : 0.8841
model : KNeighborsClassifier() and accuracy score is : 0.8841

```

	Accuracy	
Logistic_scl	0.8804	
LinearDiscriminant_scl	0.8696	
SVM_scl	0.8841	
KNeighbors_scl	0.8841	

- SVC: 정확도 약 72.46% -> 약 88.41%
- KNN: 정확도 약 71.74% -> 약 88.41%
- 예상대로 스케일링을 적용한 결과, KNN과 SVM 모두 이전보다 더 좋은 성능을 보임

(4) Ensemble Models (AdaBoost & Gradient Boosting & Random Forest & Extra Trees)

```

accuracy = []
model_names = []

X= df.drop('HeartDisease', axis=1)
y= df['HeartDisease']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

ohe= OneHotEncoder()
ct= make_column_transformer((ohe,categorical),remainder='passthrough')

ada = AdaBoostClassifier(random_state=0)
gb = GradientBoostingClassifier(random_state=0)
rf = RandomForestClassifier(random_state=0)
et= ExtraTreesClassifier(random_state=0)

models = [ada,gb,rf,et]

for model in models:
    pipe = make_pipeline(ct, model)
    pipe.fit(X_train, y_train)
    y_pred = pipe.predict(X_test)
    accuracy.append(round(accuracy_score(y_test, y_pred),4))
    print (f'model : {model} and accuracy score is : {round(accuracy_score(y_test, y_pred),4)}')

model_names = ['Ada','Gradient','Random','ExtraTree']
result_df3 = pd.DataFrame({'Accuracy':accuracy}, index=model_names)
result_df3

```

```

model : AdaBoostClassifier(random_state=0) and accuracy score is : 0.8732
model : GradientBoostingClassifier(random_state=0) and accuracy score is : 0.8768
model : RandomForestClassifier(random_state=0) and accuracy score is : 0.8877
model : ExtraTreesClassifier(random_state=0) and accuracy score is : 0.8804

```

Accuracy 	
Ada	0.8732
Gradient	0.8768
Random	0.8877
ExtraTree	0.8804

- AdaBoostClassifier는 정확도 약 87.32%
- GradientBoostingClassifier는 정확도 약 87.68%
- RandomForestClassifier는 정확도 약 88.77%
- ExtraTreesClassifier는 정확도 약 88.04%
- 앙상블 모델들의 정확도는 서로 큰 차이 없이 비슷하게 나타남
- 특히 Random Forest와 Extra Trees는 유사한 정확도를 보임
- 두 모델 모두 하이퍼파라미터 튜닝을 통해 성능을 추가로 개선 가능

(5) Famous Trio (XGBoost & LightGBM & Catboost)

```

accuracy = []
model_names = []

X= df.drop('HeartDisease', axis=1)
y= df['HeartDisease']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

ohe= OneHotEncoder()
ct= make_column_transformer((ohe,categorical),remainder='passthrough')

xgbc = XGBClassifier(random_state=0)
lgbmc = LGBMClassifier(random_state=0, verbose=-1)

models = [xgbc, lgbmc]

for model in models:
    pipe = make_pipeline(ct, model)
    pipe.fit(X_train, y_train)
    y_pred = pipe.predict(X_test)
    accuracy.append(round(accuracy_score(y_test, y_pred),4))

model_names = ['XGBoost', 'LightGBM']
result_df4 = pd.DataFrame({'Accuracy':accuracy}, index=model_names)
result_df4

```

/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:2739: UserWarning:

X does not have valid feature names, but LGBMClassifier was fitted with feature names

	Accuracy	
XGBoost	0.8478	
LightGBM	0.8732	

- XGBoost는 정확도 약 84.78%
- LightGBM는 정확도 약 87.32%

(6) CATBOOST

- **CatBoost**: 범주형 변수 처리에 특화된 Gradient Boosting 모델

```

accuracy = []
model_names = []

X= df.drop('HeartDisease', axis=1)
y= df['HeartDisease']
categorical_features_indices = np.where(X.dtypes == 'object')[0]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

model = CatBoostClassifier(verbose=False, random_state=0)

model.fit(X_train, y_train, cat_features=categorical_features_indices, eval_set=(X_test, y_test))
y_pred = model.predict(X_test)
accuracy.append(round(accuracy_score(y_test, y_pred), 4))

model_names = ['Catboost_default']
result_df5 = pd.DataFrame({'Accuracy': accuracy}, index=model_names)
result_df5

```

Accuracy		
Catboost_default	0.8913	

- CatBoost는 정확도 약 89.13%
- 기본 하이퍼파라미터를 사용했을 때, CatBoost가 다른 두 모델(XGBoost, LightGBM)보다 더 좋은 성능을 보임

(7) Catboost HyperParameter Tuning with Optuna

(7)-1. 최적 하이퍼파라미터 탐색

```

def objective(trial):
    X= df.drop('HeartDisease', axis=1)
    y= df['HeartDisease']
    categorical_features_indices = np.where(X.dtypes == 'object')[0]

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

    param = {
        "objective": trial.suggest_categorical("objective", ["Logloss", "CrossEntropy"]),
        "colsample_bylevel": trial.suggest_float("colsample_bylevel", 0.01, 0.1),
        "depth": trial.suggest_int("depth", 1, 12),
        "boosting_type": trial.suggest_categorical("boosting_type", ["Ordered", "Plain"]),
        "bootstrap_type": trial.suggest_categorical(
            "bootstrap_type", ["Bayesian", "Bernoulli", "MVS"]
        ),
        "used_ram_limit": "3gb",
    }

    if param["bootstrap_type"] == "Bayesian":
        param["bagging_temperature"] = trial.suggest_float("bagging_temperature", 0, 10)
    elif param["bootstrap_type"] == "Bernoulli":
        param["subsample"] = trial.suggest_float("subsample", 0.1, 1)

    cat_cls = CatBoostClassifier(**param)

    cat_cls.fit(X_train, y_train, eval_set=(X_test, y_test), cat_features=categorical_features_indices, verbose=0, early_stopping_rounds=100)

    preds = cat_cls.predict(X_test)
    pred_labels = np.rint(preds)
    accuracy = accuracy_score(y_test, pred_labels)
    return accuracy

if __name__ == "__main__":
    study = optuna.create_study(direction="maximize")
    study.optimize(objective, n_trials=50, timeout=600)

    print("Number of finished trials: {}".format(len(study.trials)))

    print("Best trial:")
    trial = study.best_trial

    print("  Value: {}".format(trial.value))

    print("  Params: ")
    for key, value in trial.params.items():
        print("    {}: {}".format(key, value))

```

```

[1 2020-10-03 07:33:33,713] Trial 43 finished
Number of finished trials: 50
Best trial:
  Value: 0.9057971014492754
  Params:
    objective: Logloss
    colsample_bylevel: 0.08678886328000703
    depth: 12
    boosting_type: Plain
    bootstrap_type: Bayesian
    bagging_temperature: 0.7691967164376945

```

- Optuna를 이용해 50회의 하이퍼파라미터 탐색을 수행
- 최고 정확도는 약 90.58%

(7)-2. 재학습

```

accuracy = []
model_names = []

X= df.drop('HeartDisease', axis=1)
y= df['HeartDisease']
categorical_features_indices = np.where(X.dtypes == 'object')[0]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

model = CatBoostClassifier(verbose=False, random_state=0,
                           objective='Logloss', colsample_bylevel=0.08679886328000703,
                           depth=12, boosting_type='Plain', bootstrap_type='Bayesian')

model.fit(X_train, y_train, cat_features=categorical_features_indices, eval_set=(X_test, y_test))
y_pred = model.predict(X_test)
accuracy.append(round(accuracy_score(y_test, y_pred), 4))
print(classification_report(y_test, y_pred))

model_names = ['Catboost_tuned']
result_df6 = pd.DataFrame({'Accuracy': accuracy}, index=model_names)
result_df6

```

...	precision	recall	f1-score	support
0	0.86	0.89	0.88	112
1	0.93	0.90	0.91	164
accuracy			0.90	276
macro avg	0.89	0.90	0.90	276
weighted avg	0.90	0.90	0.90	276

Accuracy 

Catboost_tuned	0.8986	
-----------------------	--------	---

- 정확도 약 89.13% -> 89.96%로 향상

(8) Feature Importance

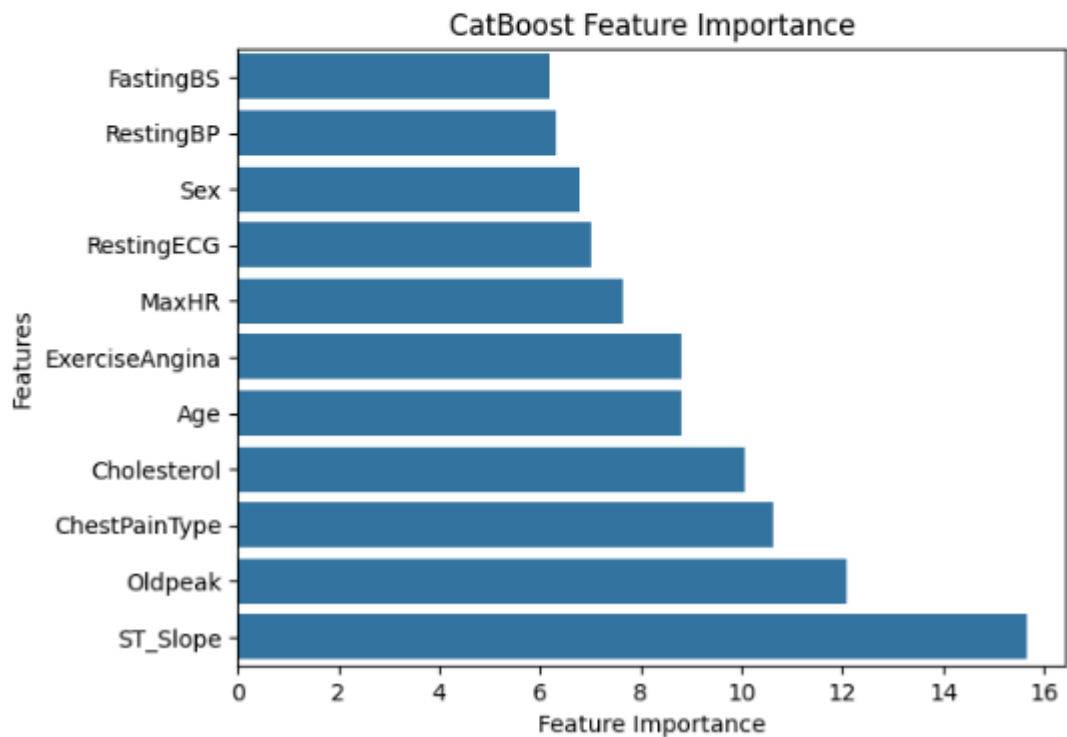
```

feature_importance = np.array(model.get_feature_importance())
features = np.array(X_train.columns)
fi={'features':features,'feature_importance':feature_importance}
df_fi = pd.DataFrame(fi)
df_fi.sort_values(by=['feature_importance'], ascending=True,inplace=True)

sns.barplot(data=df_fi, x='feature_importance', y='features')

plt.title('CatBoost Feature Importance')
plt.xlabel('Feature Importance')
plt.ylabel('Features')
plt.show()

```



- ST_Slope 가 가장 중요한 변수로 나타났으며, 그다음으로 oldpeak , ChestPainType , Cholesterol 순으로 높은 중요도를 보임

(9) Model Comparison

```

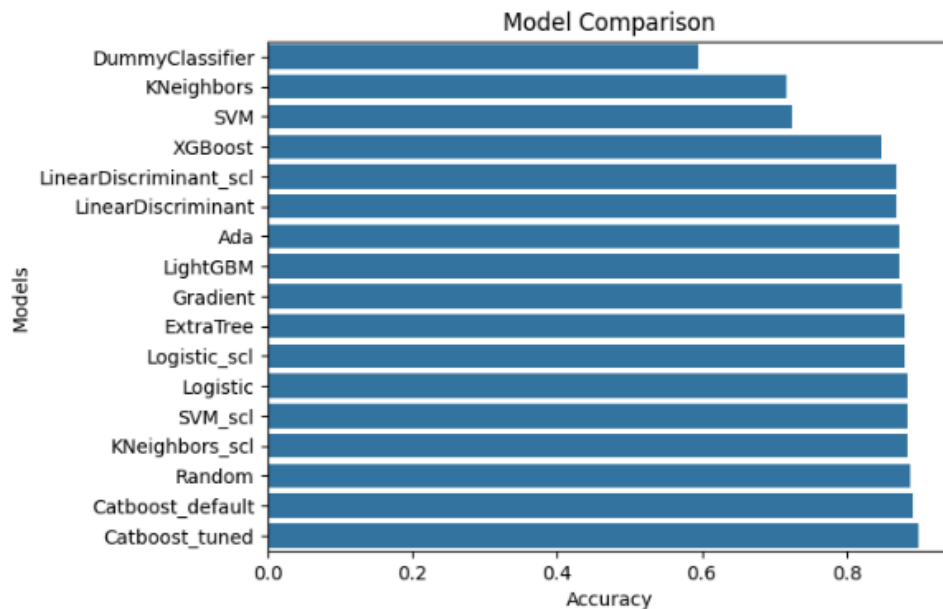
result_final = pd.concat([dummy_result_df, result_df1, result_df2, result_df3, result_df4, result_df5, result_df6], axis=0)

result_final.sort_values(by='Accuracy', ascending=True, inplace=True)

sns.barplot(data=result_final, x='Accuracy', y=result_final.index)

plt.title('Model Comparison')
plt.xlabel('Accuracy')
plt.ylabel('Models')
plt.show()

```



- DummyClassifier 가 가장 낮은 성능을 보임
- KNN 과 SVM 은 스케일링 적용 후 성능이 크게 향상 - 앙상블 모델들은 전반적으로 유사한 성능을 보였으며, 최종적으로 Optuna 를 통해 하이퍼파라미터를 조정한 CatBoost 모델이 약 89.86%로 가장 높은 정확도를 기록

3. Conclusion

- 심장질환 여부를 분류하기 위한 머신러닝 모델을 구축
- 먼저 데이터에 대한 탐색적 데이터 분석(EDA)을 수행
- 타겟 변수의 분포를 확인하고 모델 평가에 사용할 평가지표로 Accuracy 를 선택
- 타겟 변수와 각 피처의 분포 및 관계를 자세히 분석
- 범주형 변수를 모델에서 사용할 수 있도록 수치형 데이터로 변환
- Pipeline 을 활용하여 전처리와 모델 학습 과정을 연결
- 다양한 분류 모델의 성능을 비교하여 문제에 적합한 모델을 탐색
- 그중 CatBoost 의 성능을 자세히 분석
- Optuna 를 이용해 CatBoost 의 하이퍼파라미터 튜닝을 수행하고 성능 변화를 확인

- 마지막으로 Feature Importance 를 통해 심장질환 예측에 중요하게 사용된 변수들을 확인



조윤서



이전 포스트

[Euron] 머신러닝 스터디 2주차

0개의 댓글

댓글을 작성하세요

댓글 작성



Powered by
Stellate